

claude-windows / f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb\subagents\workflows\wf_35b735a9-0a9\agent-a0b9fa24b3f3236ed.jsonl`  
- SHA-256: `70d9e936bc610ff30a97462ded990327aa46edeba8576f7ac71c89920921fb04`  
- Source modified: `2026-06-21T15:03:16+00:00`  
- Imported at: `2026-07-05T16:48:22+00:00`  
- project: `wf_35b735a9-0a9`  
- session_id: `f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb`
```

Transcript

1. user (2026-06-21T14:59:20.111Z)

Review the UNCOMMITTED change on branch serving/exposure-safety-preflight in C:\\Users\\avidu\\Projects\\badminton-highlight-indexer.

WHAT CHANGED: backend/config/startup.py gained an exposure-safety posture. A new `_exposure_checks(config)` returns findings gated on `security.edge_auth_enabled` (default `False => []` strict no-op). When ON: `EXPOSED_NO_VIDEO_ROOT` (ERROR if `security.allowed_video_root` unset/blank), `EXPOSED_EDGE_AUTH_INCOMPLETE` (ERROR if `cf_team_domain` or `cf_access_aud` missing), `EXPOSED_AUTH_EXTRA_MISSING` (WARN if PyJWT not importable via `_auth_extra_available()`).

`run_startup_checks` gained `include_exposure` (default `False`); `enforce_startup_checks` threads it; backend/main.py `_enforce_startup_or_exit` passes `include_exposure=True` (server only); the worker (backend/worker/___main__.py `_run_startup_checks`) does NOT pass it. Tests in `tests/test_startup_checks.py`. Tracker/changelog in docs/COMPUTE_DECOUPLED_SERVING/README.md. Read the actual files (git diff is uncommitted in the working tree). For supporting facts, also read backend/api/auth.py (`resolve_tenant`), backend/main.py (the `/api/process` `allowed_video_root` enforcement ~line 294-311, the `static/highlights/compile` routes), backend/config/models.py `SecurityConfig` (~line 280-306).

Run 'git diff' in C:\\Users\\avidu\\Projects\\badminton-highlight-indexer to see the exact change. Be adversarial: prefer finding a real hole over rubber-stamping. Default to a finding if uncertain.

LENS = DISCIPLINE / QUALITY / TEST-COMPLETENESS. Judge against the repo conventions (M5/M8/M9 changelog style, default-OFF discipline, server-vs-worker scoping, message quality):

1. Test gaps: are all branches covered? Is there a test that the WARN (auth-extra) does NOT block server start (only the two ERRORS exit)? Is there a test that `enforce_startup_checks` LOGS the exposure errors at ERROR before raising (mirrors `test_enforce_raises_on_fatal`)? Is the 'fully configured exposure does not exit' path covered? Any missing edge case (e.g. `edge_auth` on + only `auth-extra-missing => server should NOT exit`)?
2. Message quality: are the three messages actionable + accurate? Does `EXPOSED_EDGE_AUTH_INCOMPLETE` duplicate the runtime check in `auth.resolve_tenant` in a way that could DRIFT (two sources of truth for 'team+aud required')? Is that acceptable or should one reference the other?
3. Is tying the exposure check to enforce via `include_exposure` the cleanest seam, or is there a simpler/idomatic approach already in the codebase? Any over-engineering?
4. Docs: does the changelog entry + M9 row note honestly state what's verified vs owner-side? Any overclaim?

Return findings with file:line + a ship/ship-with-fixes/block bottom line.

2. assistant (2026-06-21T14:59:22.179Z)

I'll start by checking the git diff to see the uncommitted changes, then review the files systematically for the issues you've outlined.

3. user (2026-06-21T14:59:22.739Z)

warning: in the working copy of 'backend/config/startup.py', LF will be replaced by CRLF the

```

next time Git touches it
warning: in the working copy of 'tests/test_startup_checks.py', LF will be replaced by CRLF the
next time Git touches it
diff --git a/backend/config/startup.py b/backend/config/startup.py
index b16f0de..28a5858 100644
--- a/backend/config/startup.py
+++ b/backend/config/startup.py
@@ -1,4 +1,4 @@
-"""Per-profile startup checks (COMPUTE_DECOUPLED_SERVING M5).
+"""Startup checks ? per-profile coherence (M5) + exposure-safety posture (M9 hardening).

A deployment profile (`truly-local` / `cloud-serving` / `cpu-mock`) implies a coherent
runtime. These checks fail/warn fast at process startup ? a loud signal before any
@@ -15,10 +15,20 @@ runtime. These checks fail/warn fast at process startup ? a loud signal
*b
    a warning: Application Default Credentials can come from the metadata server / gcloud,
which
    we can't probe offline, so the storage backend's own init is the authority; we just flag
it.

-DEFAULT-OFF: with no profile selected (`profile == "") this is a strict no-op ?
exactly
-the pre-M5 behaviour. Only an explicitly-selected profile triggers any check (mirrors the
loader,
-which applies a preset only for an explicit profile). So nothing changes for today's manual-
knob
-deployments.
+Exposure-safety posture (server only, `include_exposure=True`). When the engine is
exposed
+behind an edge gate (the owner flips `security.edge_auth_enabled` on for the Cloudflare-
Access
+boundary ? the alpha-shareable "tunnel-to-a-box" topology), the gate is only half the safety
story.
+A half-configured exposure starts looking healthy then leaks / 500s per request, so we verify
the
+rest of the posture at boot: the `allowed_video_root` path-jail is set (else an
authenticated
+tenant could read an arbitrary local file ? error), the CF team-domain + AUD are present
(else
+no token can verify ? error, lifting the current first-request failure to boot), and the
+`auth` extra is importable (else every request 500s ? warning). Only the HTTP server
passes
+`include_exposure=True`; the worker reads `--video-uri` (not untrusted tenant paths), so it
is
+never "exposed" and stays unaffected.
+
+DEFAULT-OFF (both dimensions): with no profile selected (`profile == "") the profile
checks are
+a strict no-op ? exactly the pre-M5 behaviour; with `edge_auth_enabled=False` (today's
default)
+the exposure checks are a strict no-op too. So nothing changes for today's manual-knob
deployments.
"""

from __future__ import annotations
@@ -66,30 +76,100 @@ def _creds_discoverable(env: Mapping[str, str]) -> bool:
    return any(env.get(k) for k in _GCP_ENV_HINTS)

+def _auth_extra_available() -> bool:
+    """True if PyJWT (the `auth` extra) is importable ? probed WITHOUT importing it
(find_spec),
+    so this stays cheap + side-effect-free. Module-level so tests can monkeypatch it
deterministically
+    (whether PyJWT is installed in the test env must not change the WARN's presence)."""

```

```

+     try:
+         import importlib.util
+         return importlib.util.find_spec("jwt") is not None
+     except Exception: # pragma: no cover - find_spec only raises on a broken parent package
+         return False
+
+
+def _exposure_checks(config: Any) -> list[StartupCheck]:
+    """Exposure-safety posture for an *exposed* HTTP server. Gated on
+    ``security.edge_auth_enabled``
+    (default False ? ``[]`` = strict no-op, today's behaviour). When edge auth is ON ? the "I
+    am
+    exposing this engine behind the CF-Access gate" signal ? the gate alone isn't enough:
+    verify the
+    rest of the posture at boot so a half-configured exposure fails fast instead of leaking /
+    500-ing
+    per request. Config-only (no IO beyond the ``find_spec`` auth-extra probe)."""
+    sec = getattr(config, "security", None)
+    if not sec or not getattr(sec, "edge_auth_enabled", False):
+        return [] # DEFAULT-OFF: edge auth off ? no posture enforcement (byte-identical to
+    today).
+
+    out: list[StartupCheck] = []
+
+    # (1) The path-jail is load-bearing once authenticated tenants can hit /api/process:
+    WITHOUT it,
+    # a tenant can point the engine at an ARBITRARY local file (allowed_video_root unset = "any
+    local
+    # file", the single-user default). Fatal for an exposed engine.
+    root = getattr(sec, "allowed_video_root", None)
+    if not root or not str(root).strip():
+        out.append(StartupCheck(
+            LEVEL_ERROR, "EXPOSED_NO_VIDEO_ROOT",
+            "security.edge_auth_enabled is ON (engine exposed) but security.allowed_video_root
+            is "
+            "unset ? an authenticated tenant could make /api/process read an arbitrary local
+            file. "
+            "Set allowed_video_root to a jail directory before exposing the engine.",
+        ))
+
+    # (2) Edge auth cannot verify a token without the team domain + AUD. Today that only fails
+    on the
+    # FIRST request (auth.resolve_tenant); lift it to BOOT so a misconfigured gate never starts
+    # "healthy" then 401s every request.
+    team = (getattr(sec, "cf_team_domain", "") or "").strip()
+    aud = (getattr(sec, "cf_access_aud", "") or "").strip()
+    if not team or not aud:
+        missing = " + ".join(n for n, v in (("cf_team_domain", team), ("cf_access_aud", aud))
+    if not v)
+        out.append(StartupCheck(
+            LEVEL_ERROR, "EXPOSED_EDGE_AUTH_INCOMPLETE",
+            f"security.edge_auth_enabled is ON but {missing} is not configured ? the Cf-Access
+            JWT "
+            "cannot be verified (every request would fail). Set the CF team domain + Access
+            AUD.",
+        ))
+
+    # (3) The auth extra (PyJWT[crypto]) is lazy-imported by auth.verify_cf_access_token. If
+    it's
+    # missing, every request 500s at verify time ? a boot WARN beats a per-request 500.
+    Warning, not
+    # fatal: the auth module's own ImportError is the final authority.
+    if not _auth_extra_available():
+        out.append(StartupCheck(
+            LEVEL_WARN, "EXPOSED_AUTH_EXTRA_MISSING",

```

```

+         "security.edge_auth_enabled is ON but PyJWT[crypto] is not importable ? Cf-Access "
+         "verification will fail at request time. Install the 'auth' extra (pip install -e
.[auth])).",
+     ))
+
+     return out
+
+
+ def run_startup_checks(
+     config: Any,
+     *,
+     cuda_available: Optional[bool] = None,
+     env: Optional[Mapping[str, str]] = None,
+     include_exposure: bool = False,
+ ) -> list[StartupCheck]:
+     """Return the per-profile findings for ``config`` (empty when no profile is selected).
+     """Return the startup findings for ``config``.

    ``cuda_available`` is the caller's best-effort GPU probe (``None`` = not probed ? GPU
checks
    are skipped; the API server passes ``None``, the worker passes
    ``presets.probe_cuda_available()``).
    - Pure: no IO, no torch import ? the caller owns the probe so this stays importable
everywhere.
+     ``include_exposure`` adds the exposure-safety posture (``_exposure_checks``) ? the HTTP
**server**
+     passes ``True``; the worker leaves it ``False`` (it is never "exposed"). Both dimensions
are
+     DEFAULT-OFF: no profile ? no profile checks; ``edge_auth_enabled=False`` ? no exposure
checks.
+     Pure: no IO (beyond the ``find_spec`` auth probe), no torch import ? the caller owns the
GPU probe.
    """
+     e = os.environ if env is None else env
+     checks: list[StartupCheck] = []
+
+     # Exposure-safety posture ? INDEPENDENT of deployment.profile (a tunnel-to-a-box exposure
often
+     # runs with no profile set), gated on edge_auth_enabled (default-OFF). Server-only.
+     if include_exposure:
+         checks.extend(_exposure_checks(config))
+
+     profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
+     if not profile:
+         return [] # DEFAULT-OFF: no profile ? no checks (identical to pre-M5).
+     return checks # no profile ? only the exposure posture (itself off unless edge auth is
on).

    - e = os.environ if env is None else env
      storage_backend = getattr(getattr(config, "storage", None), "backend", "local")
      indexing = getattr(config, "indexing", None)
      detector_impl = getattr(indexing, "detector_impl", "auto")
      device = getattr(getattr(indexing, "wasb", None), "device", "cuda")

    - checks: list[StartupCheck] = []
    -
      if profile == "truly-local":
          # truly-local reads bytes from the local FS or a *mounted* Drive ? a cloud backend here
is
          # an inconsistent (but not fatal) choice worth surfacing.
@@ -162,15 +242,18 @@ def enforce_startup_checks(
    cuda_available: Optional[bool] = None,
    env: Optional[Mapping[str, str]] = None,
    raise_on_error: bool = True,
+     include_exposure: bool = False,

```

```

    logger: Optional[logging.Logger] = None,
) -> list[StartupCheck]:
    """Run the checks, LOG each (errors at ERROR, warnings at WARNING), and ? unless
    ``raise_on_error=False`` ? raise ``StartupCheckError`` if any FATAL check tripped.

- Returns the full findings list (so a caller can inspect/surface them). A no-op with no
logging
- and no raise when no profile is selected (``run_startup_checks`` returns ``[]``)."""
+ ``include_exposure`` (server-only) adds the exposure-safety posture. Returns the full
findings
+ list (so a caller can inspect/surface them). A no-op with no logging and no raise when both
+ dimensions are off (``run_startup_checks`` returns ``[]``)."""
    log = logger or logging.getLogger(__name__)
- checks = run_startup_checks(config, cuda_available=cuda_available, env=env)
+ checks = run_startup_checks(
+     config, cuda_available=cuda_available, env=env, include_exposure=include_exposure)
    for c in checks:
        if c.level == LEVEL_ERROR:
            log.error("[STARTUP] %s: %s", c.code, c.message)
diff --git a/backend/main.py b/backend/main.py
index 6768d30..0aec8e3 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -562,13 +562,15 @@ def run_cli_diagnose_clip(video_path: str, timestamp: float):

def _enforce_startup_or_exit(config) -> None:
- """M5: run the per-profile startup checks for the server/CLI path and ``sys.exit(1)`` on a
- FATAL incoherent profile (e.g. cloud-serving pointed at local-only storage) BEFORE any
work.
- A strict no-op when no ``deployment.profile`` is selected (today's behaviour). No GPU probe
- here ? the server stays torch-light; the detector healthcheck is the GPU authority.
Extracted
- so the fail-fast wiring + the default-OFF no-op are unit-testable."""
+ """M5: run the startup checks for the server path and ``sys.exit(1)`` on a FATAL finding
BEFORE
+ any work ? a fatal incoherent profile (e.g. cloud-serving pointed at local-only storage)
OR, for
+ the exposed server (``include_exposure=True``, M9 hardening), a half-configured exposure
(edge
+ auth on without the ``allowed_video_root`` jail / CF team+AUD). A strict no-op when no
+ ``deployment.profile`` is selected AND edge auth is off (today's behaviour). No GPU probe
here ?
+ the server stays torch-light; the detector healthcheck is the GPU authority. Extracted so
the
+ fail-fast wiring + the default-OFF no-op are unit-testable."""
    try:
-         enforce_startup_checks(config, logger=logger)
+         enforce_startup_checks(config, logger=logger, include_exposure=True)
    except StartupCheckError as e:
        logger.error("[STARTUP] refusing to start: %s", e)
        sys.exit(1)
diff --git a/docs/COMPUTE_DECOUPLED_SERVING/README.md b/docs/COMPUTE_DECOUPLED_SERVING/README.md
index 3b3b1c5..f59f5d9 100644
--- a/docs/COMPUTE_DECOUPLED_SERVING/README.md
+++ b/docs/COMPUTE_DECOUPLED_SERVING/README.md
@@ -121,7 +121,7 @@ Legend: ? Todo · ? In progress · ? Done · ? Gated. **One small PR per
| M6 | Cloud Run GPU **dispatch shim** (control plane ? job via storage ? re-claim) +
**containerized worker image** (ffmpeg+CUDA+WASB+fonts); mocked-dispatch tests | M5 | ? owner
(GCP spend) | ? | [#287](https://github.com/Khelsutra/badminton-highlight-indexer/pull/287)
(shim+image+mocked ?; real build/run = M7 ? owner) |
| M7 | **cloud-serving profile** e2e on L4: upload<2GB` + gdrive/bucket ? detect+stitch on
Cloud Run ? reel via signed URL; **DEPLOY_REPORT** (posterity, sample runs + contact sheet) | M6
| ? owner (spend) | ? | ? |
| M8 | Per-job **cost ledger** (v6 migration: `owner_id, gpu_active_seconds, wall_seconds,
```

bytes_out, cost_usd, cost_breakdown_json`) + **pricebook** + aggregation + `/api/.../cost` | M7
| ? owner (billing) | ? | [#288](https://github.com/Khelsutra/badminton-highlight-indexer/pull/288) (ledger+pricebook+/cost ? default-OFF; real prices + usage-emission ? owner/follow-up) |
-| M9 | **Edge auth** (Cf-Access extraction) + per-tenant scoping on `user\_id`; DPA/consent gate wiring | M7 | ? owner (privacy/counsel) | ? | [#290](https://github.com/Khelsutra/badminton-highlight-indexer/pull/290) (auth: Cf-Access JWT verify + owner_id tag, default-OFF ?; consent cols + ToS/DPA ? owner/counsel) |
+| M9 | **Edge auth** (Cf-Access extraction) + per-tenant scoping on `user\_id`; DPA/consent gate wiring | M7 | ? owner (privacy/counsel) | ? | [#290](https://github.com/Khelsutra/badminton-highlight-indexer/pull/290) (auth: Cf-Access JWT verify + owner_id tag, default-OFF ?; **exposure-safety boot posture** in `startup.py` ? edge-auth-on requires the `allowed\_video\_root` jail + CF team/AUD or the server refuses to start, see changelog; consent cols + ToS/DPA ? owner/counsel) |
M10	`byo\_worker` / zero-infra-BYO ? **design-only, DEFERRED**	M8,M9	? explicit owner approval	?	?
M11	**Data-flywheel harness** (D12): **capture** the upload + Gemini reel/rally output ? **reliably store** under the per-video workspace ? **shadow-eval** owned-vs-Gemini (dual-eval-set / `experiment.compare`) ? **promote** good ones to the golden TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the owned model climbs to the D11 floor. Reuses `data\_pipeline/` + `backend/eval/` + the consent gate (M9).	M7,M9	? owner (consent/privacy)	?	?
M12	**gdrive-api** (OAuth) StorageBackend (D14 Northstar): **cloud reads the source** from + **writes the stitched reel back** to the user's Google Drive next to the source (per-user OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local mount backend; slots into the `StorageBackend` ABC.	M7,M9	? owner (OAuth/consent)	?	?
@@ -167,3 +167,4 @@ Legend: ? Todo · ? In progress · ? Done · ? Gated. **One small PR per**
- 2026-06-21 ? **M8 code landed** ([#288](https://github.com/Khelsutra/badminton-highlight-indexer/pull/288)): per-job **cost ledger** ? `backend/billing.py` (pure `usage × pricebook` math, USD internal / INR display) + a **v6 migration** (NULLABLE cost columns on `jobs` + a versioned `pricebook` table seeded `placeholder-v0` at \$0; additive-NUL, idempotent) + `record\_job\_cost`/`get\_job\_cost`/`get\_video\_cost` + `GET /api/jobs/{id}/cost` + `/api/videos/{id}/cost` + `BillingConfig` (**default-OFF**) + a gated `\_maybe\_record\_job\_cost` hook. Adversarial review (`wf\_498b24af`) folded a **major silent-GPU-under-bill** (an unpriced `gpu\_type` is now surfaced, not invisible \$0) + the **honest no-op** caveat (usage emission is a follow-up ? cost 0 until wired) + breakdown reconciliation. Suite green, ruff+mypy clean. **M8** ? ? owner inserts real GCP prices (a new pricebook version) + the worker usage-emission is a follow-up; placeholder ships \$0 = no user-facing change. v6 is the shared bump (M9-consent cols extend it).
- 2026-06-21 ? **M6 code landed** ([#287](https://github.com/Khelsutra/badminton-highlight-indexer/pull/287)): `backend/compute/` ComputeBackend registry ? `CloudRunCompute` dispatches a Cloud Run **Job** (injectable client) then **bucket-polls** a done-marker via `execution\_policy.poll\_until` + the storage layer (D16); `get\_compute\_backend` returns a backend ONLY for `compute\_target=cloud\_run` (else `None` ? the worker's in-process path, **default-OFF**). The dispatched container runs **inline** (forced `local\_gpu`+native, never re-dispatch); the worker writes a `--done-uri` completion marker on success. `deploy/cloudrun/{Dockerfile,README}` = the L4 image (CUDA+ffmpeg+fonts+`wasb` conda env; weights staged per WASB-C3) + the M7 runbook. Adversarial review (`wf\_87b22fa4`) folded a **blocker** (cloud `fetch` needs a dst ? reads silently returned `{}` on gcs/s3) + a poll-wait test gap + `PolicyTimeout`?clean rc 4. Suite green, ruff+mypy clean. **M6** ? ? the real build/push/L4 run + DEPLOY_REPORT is M7 (owner, D17 budget).
- 2026-06-21 ? **M5 code landed** ([#286](https://github.com/Khelsutra/badminton-highlight-indexer/pull/286)): `backend/config/startup.py` ? per-profile fail/warn-fast startup checks (cloud-serving + non-cloud storage = the one fatal coherence error; GPU/creds mismatches = warnings, the runner healthcheck stays the authority), wired into the server (`\_enforce\_startup\_or\_exit` ? exit 1) + the worker (`cmd\_infer`/`cmd\_train` ? rc 2), default-OFF (probe gated behind an active profile ? the worker stays torch-free at `profile=""`). **L4 profile-parity test** (`tests/test\_profile\_parity.py`): same stub trajectory ? byte-identical windows + segment ranges across truly-local / cpu-mock / no-profile + worker-vs-in-process. Adversarial 4-lens review folded (`wf\_34671ac1`). Suite green, ruff+mypy clean. **M5** ? ? the real-GPU truly-local runlog (L5) is owner-side (template pre-staged in `runlogs/`).
+- 2026-06-21 ? **M9 hardening: exposure-safety boot posture** (alpha-shareable "tunnel-to-a-box" enabler): extended `backend/config/startup.py` with a **server-only exposure posture** (new `include\_exposure=True` from `\_enforce\_startup\_or\_exit`; the worker stays unaffected ? it reads `--video-uri`, not untrusted tenant paths). Gated on `security.edge\_auth\_enabled` (**default-OFF**

? strict no-op**, byte-identical to today): when the owner flips edge auth ON to expose the engine behind the CF-Access gate, three checks fire at **boot** ? `EXPOSED\_NO\_VIDEO\_ROOT` (**fatal*: `allowed\_video\_root` jail unset ? a tenant could read an arbitrary local file), `EXPOSED\_EDGE\_AUTH\_INCOMPLETE` (**fatal*: CF team-domain/AUD missing ? lifts the current first-request 401 to boot), `EXPOSED\_AUTH\_EXTRA\_MISSING` (**warn*: `PyJWT[crypto]` not importable ? would 500 per request). So a half-configured exposure fails fast instead of leaking/500-ing. +13 unit tests (default-OFF no-op, worker-ignores, both-fatals-compose-with-profile-checks, server-exit). Pairs with the appliance **T3** network posture (raw `:8000` refused ? owner-run). Suite green, ruff+mypy clean. **? backstops M9 ? the actual exposure flip + CF team/AUD/ingress + key rotation stay owner-side.**

```
diff --git a/tests/test_startup_checks.py b/tests/test_startup_checks.py
```

```
index be250a5..95b9eba 100644
```

```
--- a/tests/test_startup_checks.py
```

```
+++ b/tests/test_startup_checks.py
```

```
@@ -9,6 +9,7 @@ import logging
```

```
import pytest
```

```
from backend.config import AppConfig
```

```
+from backend.config import startup as startup_mod
```

```
from backend.config.startup import (
```

```
    LEVEL_ERROR,
```

```
    LEVEL_WARN,
```

```
@@ -29,6 +30,19 @@ def _cfg(profile="", backend="local", device="cuda", detector_impl="auto"):
    }
```

```
+def _exposed_cfg(*, edge_auth=True, video_root="/jail",
```

```
team="https://acme.cloudflareaccess.com",
```

```
+    aud="abc123", profile="", backend="local"):
```

```
+    """An AppConfig with the security/exposure fields set, for the exposure-posture checks."""
```

```
+    sec = {"edge_auth_enabled": edge_auth, "cf_team_domain": team, "cf_access_aud": aud}
```

```
+    if video_root is not None:
```

```
+        sec["allowed_video_root"] = video_root
```

```
+    return AppConfig.model_validate({
```

```
+        "deployment": {"profile": profile},
```

```
+        "storage": {"backend": backend},
```

```
+        "security": sec,
```

```
+    })
```

```
+
```

```
+
```

```
def _codes(checks):
```

```
    return {c.code for c in checks}
```

```
@@ -224,3 +238,110 @@ def test_server_enforce_does_not_exit_on_warnings():
```

```
    """A warning-only profile (truly-local + cloud storage) must NOT abort the server."""
```

```
    import backend.main as bmain
```

```
    assert bmain._enforce_startup_or_exit(_cfg(profile="truly-local", backend="gcs")) is None
```

```
+
```

```
+
```

```
+# ----- exposure-safety posture (M9 hardening, server-only)
```

```
+
```

```
+def test_exposure_off_by_default_is_noop():
```

```
+    """edge_auth_enabled=False (today's default) ? ZERO exposure checks, even with
```

```
include_exposure
```

```
+    and an otherwise-unsafe config (no allowed_video_root). The strict default-OFF guarantee."""
```

```
+    cfg = _exposed_cfg(edge_auth=False, video_root=None, team="", aud="")
```

```
+    assert run_startup_checks(cfg, include_exposure=True, env={}) == []
```

```
+
```

```
+
```

```
+def test_exposure_not_run_without_flag(monkeypatch):
```

```
+    """The worker path (include_exposure=False, the default) NEVER runs exposure checks ? even
```

```
with
```

```
+    edge auth ON and an unsafe config. Only the HTTP server opts in."""
```

```

+ monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
+ cfg = _exposed_cfg(edge_auth=True, video_root=None, team="", aud="")
+ assert run_startup_checks(cfg, env={}) == [] # include_exposure defaults False
+
+
+def test_exposed_without_video_root_is_fatal(monkeypatch):
+ monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
+ cfg = _exposed_cfg(video_root=None) # team+aud set, jail missing
+ checks = run_startup_checks(cfg, include_exposure=True, env={})
+ assert _codes(checks) == {"EXPOSED_NO_VIDEO_ROOT"}
+ assert checks[0].level == LEVEL_ERROR
+
+
+@pytest.mark.parametrize("video_root", [None, "", " "])
+def test_exposed_blank_video_root_is_fatal(monkeypatch, video_root):
+ """None, empty, and whitespace-only roots are all treated as 'unset' ? fatal."""
+ monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
+ cfg = _exposed_cfg(video_root=video_root)
+ assert "EXPOSED_NO_VIDEO_ROOT" in _codes(run_startup_checks(cfg, include_exposure=True,
+ env={}))
+
+
+@pytest.mark.parametrize("team,aud,expect_missing", [
+ ("", "abc", "cf_team_domain"),
+ ("https://t.cloudfareaccess.com", "", "cf_access_aud"),
+ ("", "", "cf_team_domain + cf_access_aud"),
+ ])
+def test_exposed_incomplete_cf_config_is_fatal(monkeypatch, team, aud, expect_missing):
+ monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
+ cfg = _exposed_cfg(team=team, aud=aud) # jail set, CF config partial
+ checks = run_startup_checks(cfg, include_exposure=True, env={})
+ assert _codes(checks) == {"EXPOSED_EDGE_AUTH_INCOMPLETE"}
+ assert checks[0].level == LEVEL_ERROR
+ assert expect_missing in checks[0].message # names exactly what's missing
+
+
+def test_exposed_missing_auth_extra_warns(monkeypatch):
+ """PyJWT not importable ? a WARN (never fatal ? the auth module's ImportError is the
+ authority)."""
+ monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: False)
+ cfg = _exposed_cfg() # fully configured otherwise
+ checks = run_startup_checks(cfg, include_exposure=True, env={})
+ assert _codes(checks) == {"EXPOSED_AUTH_EXTRA_MISSING"}
+ assert checks[0].level == LEVEL_WARN
+
+
+def test_exposed_fully_configured_is_clean(monkeypatch):
+ monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
+ cfg = _exposed_cfg() # edge auth on, jail set, team+aud set, auth extra present
+ assert run_startup_checks(cfg, include_exposure=True, env={}) == []
+
+
+def test_exposed_both_errors_together(monkeypatch):
+ """An unsafe exposure trips BOTH fatal checks at once (jail missing + CF config
+ missing)."""
+ monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
+ cfg = _exposed_cfg(video_root=None, team="", aud="")
+ codes = _codes(run_startup_checks(cfg, include_exposure=True, env={}))
+ assert codes == {"EXPOSED_NO_VIDEO_ROOT", "EXPOSED_EDGE_AUTH_INCOMPLETE"}
+
+
+def test_exposure_composes_with_profile_checks(monkeypatch):
+ """Exposure posture is INDEPENDENT of deployment.profile ? an exposed cloud-serving config
+ with
+ local storage trips both the exposure jail error AND the profile STORAGE_INCOHERENT

```



```

error."""
+ monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
+ cfg = _exposed_cfg(video_root=None, profile="cloud-serving", backend="local")
+ codes = _codes(run_startup_checks(cfg, cuda_available=True, include_exposure=True,
+                               env={"K_SERVICE": "x"}))
+ assert codes == {"EXPOSED_NO_VIDEO_ROOT", "STORAGE_INCOHERENT"}
+
+
+def test_server_enforce_exits_on_unsafe_exposure(monkeypatch):
+ """The server's fail-fast wiring aborts (exit 1) when edge auth is on but the engine is
+ exposed
+ without the allowed_video_root jail ? BEFORE serving a single request."""
+ monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
+ import backend.main as bmain
+ with pytest.raises(SystemExit) as ei:
+     bmain._enforce_startup_or_exit(_exposed_cfg(video_root=None))
+ assert ei.value.code == 1
+
+
+def test_server_enforce_allows_safe_exposure(monkeypatch):
+ """A fully-configured exposure starts cleanly (no exit)."""
+ monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
+ import backend.main as bmain
+ assert bmain._enforce_startup_or_exit(_exposed_cfg()) is None
+
+
+def test_worker_path_ignores_unsafe_exposure(monkeypatch, tmp_path):
+ """The worker (cmd path) does NOT enforce exposure posture ? an edge-auth-on config that
+ would
+ abort the server runs fine on the worker (it reads --video-uri, not untrusted tenant
+ paths)."""
+ monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
+ from backend.worker import __main__ as wmain
+ cfg_file = tmp_path / "c.json"
+ cfg_file.write_text('{"security": {"edge_auth_enabled": true}}') # exposed-ish, no
+ jail/team/aud
+ assert wmain._run_startup_checks(wmain.load_config(str(cfg_file))) is True

```

4. assistant (2026-06-21T14:59:24.653Z)

Now let me read the full context of the relevant files to understand the implementation and evaluate it.

5. user (2026-06-21T14:59:25.509Z)

```

1      """Startup checks ? per-profile coherence (M5) + exposure-safety posture (M9 hardening).
2
3      A deployment profile (`truly-local` / `cloud-serving` / `cpu-mock`) implies a
coherent
4      runtime. These checks fail/warn fast at process startup ? a loud signal *before* any
5      (expensive) job runs ? when the active profile is inconsistent with the environment:
6
7      * cloud-serving with a non-cloud storage backend ? a hard **error** (the one
config bug we
8      can detect with certainty + zero false-positive risk: a cloud profile that can't
reach a
9      bucket can never serve). This is the README §4.3 "loud error before any job".
10     * a GPU mismatch (e.g. `device='cuda'` on a box where CUDA isn't visible, or
`device='auto'`
11     falling back to CPU) ? a **warning**, never an error: the probe is best-effort
(torch may live
12     only in the detector subprocess env ? see `presets.probe_cuda_available`), and the
detector's
13     own healthcheck is the real authority. A warning surfaces it early without false-
failing.

```

```

14      * **creds that can't be confirmed** (no ``GOOGLE_APPLICATION_CREDENTIALS`` / not on
Cloud Run) ?
15      a **warning**: Application Default Credentials can come from the metadata server /
gcloud, which
16      we can't probe offline, so the storage backend's own init is the authority; we just
flag it.
17
18      **Exposure-safety posture (server only, ``include_exposure=True``).** When the engine is
*exposed*
19      behind an edge gate (the owner flips ``security.edge_auth_enabled`` on for the
Cloudflare-Access
20      boundary ? the alpha-shareable "tunnel-to-a-box" topology), the gate is only half the
safety story.
21      A half-configured exposure starts *looking* healthy then leaks / 500s per request, so we
verify the
22      rest of the posture at **boot**: the ``allowed_video_root`` path-jail is set (else an
authenticated
23      tenant could read an arbitrary local file ? **error**), the CF team-domain + AUD are
present (else
24      no token can verify ? **error**, lifting the current first-request failure to boot), and
the
25      ``auth`` extra is importable (else every request 500s ? **warning**). Only the HTTP
**server** passes
26      ``include_exposure=True``; the worker reads ``--video-uri`` (not untrusted tenant
paths), so it is
27      never "exposed" and stays unaffected.
28
29      **DEFAULT-OFF (both dimensions):** with no profile selected (``profile == ""``) the
profile checks are
30      a strict **no-op** ? exactly the pre-M5 behaviour; with ``edge_auth_enabled=False``
(today's default)
31      the exposure checks are a strict **no-op** too. So nothing changes for today's manual-
knob deployments.
32      ""
33
34      from __future__ import annotations
35
36      import logging
37      import os
38      from dataclasses import dataclass
39      from typing import Any, Mapping, Optional
40
41      LEVEL_ERROR = "error"
42      LEVEL_WARN = "warn"
43
44      # Storage backends that count as "a cloud bucket" for cloud-serving coherence.
45      _CLOUD_STORAGE = ("gcs", "s3")
46      # STRONG env signals that Google ADC *might* resolve. Deliberately NOT
GOOGLE_CLOUD_PROJECT /
47      # CLOUDSDK_CONFIG ? those are often set without usable creds (a plain project id / a
config dir
48      # with no active login), so they would suppress the heads-up falsely. (s3 creds are
intentionally
49      # left to boto3's default chain ? no parallel heads-up, by design.)
50      _GCP_ENV_HINTS = ("GOOGLE_APPLICATION_CREDENTIALS", "K_SERVICE", "GCE_METADATA_HOST")
51
52
53      @dataclass(frozen=True)
54      class StartupCheck:
55          """One per-profile finding. ``level`` is ``error`` (fatal) or ``warn`` (heads-
up)."""
56
57          level: str
58          code: str          # stable, greppable code (e.g. "STORAGE_INCOHERENT")
59          message: str

```

```

60
61
62     class StartupCheckError(RuntimeError):
63         """Raised by ``enforce_startup_checks`` when an active profile fails a FATAL
check."""
64
65         def __init__(self, failures: list[StartupCheck]):
66             self.failures = failures
67             super().__init__(
68                 "deployment profile startup check failed: "
69                 + "; ".join(f"[{c.code}] {c.message}" for c in failures)
70             )
71
72
73     def _creds_discoverable(env: Mapping[str, str]) -> bool:
74         """True if *some* Google ADC source is hinted by the environment. Best-effort ? a
False here
75         only downgrades to a WARNING, never a hard failure (the storage init is the
authority)."""
76         return any(env.get(k) for k in _GCP_ENV_HINTS)
77
78
79     def _auth_extra_available() -> bool:
80         """True if PyJWT (the ``auth`` extra) is importable ? probed WITHOUT importing it
(find_spec),
81         so this stays cheap + side-effect-free. Module-level so tests can monkeypatch it
deterministically
82         (whether PyJWT is installed in the test env must not change the WARN's presence)."""
83         try:
84             import importlib.util
85             return importlib.util.find_spec("jwt") is not None
86         except Exception: # pragma: no cover - find_spec only raises on a broken parent
package
87             return False
88
89
90     def _exposure_checks(config: Any) -> list[StartupCheck]:
91         """Exposure-safety posture for an *exposed* HTTP server. Gated on
``security.edge_auth_enabled``
92         (default False ? ``[]`` = strict no-op, today's behaviour). When edge auth is ON ?
the "I am
93         exposing this engine behind the CF-Access gate" signal ? the gate alone isn't
enough: verify the
94         rest of the posture at boot so a half-configured exposure fails fast instead of
leaking / 500-ing
95         per request. Config-only (no IO beyond the ``find_spec`` auth-extra probe)."""
96         sec = getattr(config, "security", None)
97         if not sec or not getattr(sec, "edge_auth_enabled", False):
98             return [] # DEFAULT-OFF: edge auth off ? no posture enforcement (byte-identical
to today).
99
100         out: list[StartupCheck] = []
101
102         # (1) The path-jail is load-bearing once authenticated tenants can hit /api/process:
WITHOUT it,
103         # a tenant can point the engine at an ARBITRARY local file (allowed_video_root unset
= "any local
104         # file", the single-user default). Fatal for an exposed engine.
105         root = getattr(sec, "allowed_video_root", None)
106         if not root or not str(root).strip():
107             out.append(StartupCheck(
108                 LEVEL_ERROR, "EXPOSED_NO_VIDEO_ROOT",
109                 "security.edge_auth_enabled is ON (engine exposed) but
security.allowed_video_root is "
110                 "unset ? an authenticated tenant could make /api/process read an arbitrary

```

```

local file. "
111         "Set allowed_video_root to a jail directory before exposing the engine.",
112         ))
113
114     # (2) Edge auth cannot verify a token without the team domain + AUD. Today that only
fails on the
115     # FIRST request (auth.resolve_tenant); lift it to BOOT so a misconfigured gate never
starts
116     # "healthy" then 401s every request.
117     team = (getattr(sec, "cf_team_domain", "") or "").strip()
118     aud = (getattr(sec, "cf_access_aud", "") or "").strip()
119     if not team or not aud:
120         missing = " + ".join(n for n, v in (("cf_team_domain", team), ("cf_access_aud",
aud)) if not v)
121         out.append(StartupCheck(
122             LEVEL_ERROR, "EXPOSED_EDGE_AUTH_INCOMPLETE",
123             f"security.edge_auth_enabled is ON but {missing} is not configured ? the Cf-
Access JWT "
124             "cannot be verified (every request would fail). Set the CF team domain +
Access AUD.",
125             ))
126
127     # (3) The auth extra (PyJWT[crypto]) is lazy-imported by
auth.verify_cf_access_token. If it's
128     # missing, every request 500s at verify time ? a boot WARN beats a per-request 500.
Warning, not
129     # fatal: the auth module's own ImportError is the final authority.
130     if not _auth_extra_available():
131         out.append(StartupCheck(
132             LEVEL_WARN, "EXPOSED_AUTH_EXTRA_MISSING",
133             "security.edge_auth_enabled is ON but PyJWT[crypto] is not importable ? Cf-
Access "
134             "verification will fail at request time. Install the 'auth' extra (pip
install -e .[auth]).",
135             ))
136
137     return out
138
139
140     def run_startup_checks(
141         config: Any,
142         *,
143         cuda_available: Optional[bool] = None,
144         env: Optional[Mapping[str, str]] = None,
145         include_exposure: bool = False,
146     ) -> list[StartupCheck]:
147         """Return the startup findings for ``config``.
148
149         ``cuda_available`` is the caller's best-effort GPU probe (``None`` = not probed ?
GPU checks
150         are skipped; the API server passes ``None``, the worker passes
``presets.probe_cuda_available``).
151         ``include_exposure`` adds the exposure-safety posture (``_exposure_checks``) ? the
HTTP **server**
152         passes ``True``; the worker leaves it ``False`` (it is never "exposed"). Both
dimensions are
153         DEFAULT-OFF: no profile ? no profile checks; ``edge_auth_enabled=False`` ? no
exposure checks.
154         Pure: no IO (beyond the ``find_spec`` auth probe), no torch import ? the caller owns
the GPU probe.
155         """
156         e = os.environ if env is None else env
157         checks: list[StartupCheck] = []
158
159         # Exposure-safety posture ? INDEPENDENT of deployment.profile (a tunnel-to-a-box

```

```

exposure often
160     # runs with no profile set), gated on edge_auth_enabled (default-OFF). Server-only.
161     if include_exposure:
162         checks.extend(_exposure_checks(config))
163
164     profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
165     if not profile:
166         return checks # no profile ? only the exposure posture (itself off unless edge
auth is on).
167
168     storage_backend = getattr(getattr(config, "storage", None), "backend", "local")
169     indexing = getattr(config, "indexing", None)
170     detector_impl = getattr(indexing, "detector_impl", "auto")
171     device = getattr(getattr(indexing, "wasb", None), "device", "cuda")
172
173     if profile == "truly-local":
174         # truly-local reads bytes from the local FS or a *mounted* Drive ? a cloud
backend here is
175         # an inconsistent (but not fatal) choice worth surfacing.
176         if storage_backend not in ("local", "gdrive"):
177             checks.append(StartupCheck(
178                 LEVEL_WARN, "STORAGE_UNUSUAL",
179                 f"profile 'truly-local' usually uses local/gdrive storage, but
storage_backend="
180                 f"'{storage_backend}'. Reads will go to a remote backend.",
181             ))
182         # GPU heads-up (warning only ? the detector healthcheck is the authority; the
probe is
183         # best-effort and may be False even when a subprocess detector env has CUDA).
184         if cuda_available is False:
185             if device == "cuda":
186                 checks.append(StartupCheck(
187                     LEVEL_WARN, "CUDA_NOT_VISIBLE",
188                     "device='cuda' but no CUDA GPU is visible to this process. If the
box truly "
189                     "has no GPU the detector healthcheck will hard-fail ? set
indexing.wasb.device="
190                     "'auto' for the CPU fallback (M4).",
191                 ))
192             elif device == "auto":
193                 checks.append(StartupCheck(
194                     LEVEL_WARN, "CPU_FALLBACK",
195                     "device='auto' and no CUDA GPU is visible to THIS process ? auto may
resolve "
196                     "to CPU (much slower). If torch lives only in the detector
subprocess env this "
197                     "can be a false alarm; the runner's DeviceContext/healthcheck is the
authority.",
198                 ))
199
200     elif profile == "cloud-serving":
201         # The ONE fatal coherence check: a cloud profile that can't reach a bucket can't
serve.
202         if storage_backend not in _CLOUD_STORAGE:
203             checks.append(StartupCheck(
204                 LEVEL_ERROR, "STORAGE_INCOHERENT",
205                 f"profile 'cloud-serving' requires a cloud storage backend "
206                 f"({'/'.join(_CLOUD_STORAGE)}), but storage_backend='{storage_backend}'.
"
207                 "A cloud job cannot read/write a local-only backend.",
208             ))
209         elif storage_backend == "gcs" and not _creds_discoverable(e):
210             checks.append(StartupCheck(
211                 LEVEL_WARN, "CREDS_UNCONFIRMED",
212                 "no GOOGLE_APPLICATION_CREDENTIALS / Cloud Run env detected; relying on

```

```

Application "
213             "Default Credentials. The GCS backend will fail loudly at init if ADC
can't resolve.",
214         ))
215         if device == "cpu":
216             checks.append(StartupCheck(
217                 LEVEL_WARN, "CLOUD_CPU_DEVICE",
218                 "profile 'cloud-serving' runs on a GPU (L4) but
indexing.wasb.device='cpu' ? "
219                 "detection will not use the GPU.",
220             ))
221         elif cuda_available is False and device in ("cuda", "auto"):
222             checks.append(StartupCheck(
223                 LEVEL_WARN, "CLOUD_CUDA_NOT_VISIBLE",
224                 "profile 'cloud-serving' expects a GPU but no CUDA GPU is visible to
this process.",
225             ))
226
227         elif profile == "cpu-mock":
228             # The mock profile is for CI/regression ? a real detector here defeats the
purpose.
229             if detector_impl != "stub":
230                 checks.append(StartupCheck(
231                     LEVEL_WARN, "MOCK_NONSTUB_DETECTOR",
232                     f"profile 'cpu-mock' normally uses detector_impl='stub', but
detector_impl="
233                     f"'{detector_impl}'. A real detector defeats the GPU-free mock.",
234                 ))
235
236         return checks
237
238
239     def enforce_startup_checks(
240         config: Any,
241         *,
242         cuda_available: Optional[bool] = None,
243         env: Optional[Mapping[str, str]] = None,
244         raise_on_error: bool = True,
245         include_exposure: bool = False,
246         logger: Optional[logging.Logger] = None,
247     ) -> list[StartupCheck]:
248         """Run the checks, LOG each (errors at ERROR, warnings at WARNING), and ? unless
249         ``raise_on_error=False`` ? raise ``StartupCheckError`` if any FATAL check tripped.
250
251         ``include_exposure`` (server-only) adds the exposure-safety posture. Returns the
full findings
252         list (so a caller can inspect/surface them). A no-op with no logging and no raise
when both
253         dimensions are off (``run_startup_checks`` returns ``[]``)."""
254         log = logger or logging.getLogger(__name__)
255         checks = run_startup_checks(
256             config, cuda_available=cuda_available, env=env,
include_exposure=include_exposure)
257         for c in checks:
258             if c.level == LEVEL_ERROR:
259                 log.error("[STARTUP] %s: %s", c.code, c.message)
260             else:
261                 log.warning("[STARTUP] %s: %s", c.code, c.message)
262         errors = [c for c in checks if c.level == LEVEL_ERROR]
263         if errors and raise_on_error:
264             raise StartupCheckError(errors)
265         return checks
266

```

```

1      """Cloudflare Access edge-auth ? verify the Cf-Access JWT, derive the tenant ``user_id``
(M9, D9).
2
3      **DEFAULT-OFF:** when ``security.edge_auth_enabled`` is False the resolver is a strict
NO-OP (returns
4      ``None`` = the local single-user) ? nothing about the existing local path changes. When
enabled, the
5      ``Cf-Access-Jwt-Assertion`` header is **required** and **verified** (RS256) against the
team JWKS ?
6      signature + exp + issuer + audience ? so a direct hit to the Cloud Run URL **can't
spooof** the
7      identity header (the explicit D9 anti-spoof requirement). Returns the verified
``user_id`` (the
8      token's ``email``, else ``sub``).
9
10     ``jwt`` (``PyJWT[crypto]``) is **lazy-imported** so the base runtime works without it
(install the
11     ``auth`` extra for hosted/multi-tenant). The JWKS fetch is **injectable**, so the whole
verifier is
12     unit-tested **offline** with an in-test RSA keypair + a fake JWKS ? no network, no
Cloudflare.
13     """
14
15     from __future__ import annotations
16
17     import json
18     import time
19     import urllib.request
20     from typing import Any, Callable, Mapping, Optional
21
22     # Cloudflare Access sends the signed assertion in this header (it terminates at the CF
edge).
23     CF_ACCESS_HEADER = "Cf-Access-Jwt-Assertion"
24     _JWKS_PATH = "/cdn-cgi/access/certs"
25     _JWKS_TTL_SEC = 600.0 # re-fetch the team JWKS at most every 10 min (keys rotate
slowly)
26
27     JwksFetcher = Callable[[str], dict]
28
29
30     class EdgeAuthError(Exception):
31         """A required Cf-Access token was missing or failed verification ? the caller
returns 401/403."""
32
33
34     # module-level JWKS cache: team_domain -> (fetched_at, jwks_dict). Production only;
tests inject.
35     _jwks_cache: dict[str, tuple[float, dict]] = {}
36
37
38     def _issuer(team_domain: str) -> str:
39         return team_domain.rstrip("/")
40
41
42     def _default_jwks_fetcher(team_domain: str) -> dict:
43         """Fetch the team's public JWKS (``<team_domain>/cdn-cgi/access/certs``). Network ?
only the
44         production path reaches here; tests pass an injected fetcher."""
45         url = _issuer(team_domain) + _JWKS_PATH
46         with urllib.request.urlopen(url, timeout=5) as resp: # noqa: S310 (https team
domain)
47             data: Any = json.load(resp)
48             if not isinstance(data, dict) or "keys" not in data:
49                 raise EdgeAuthError(f"unexpected JWKS shape from {url}")
50             return data

```

```

51
52
53     def _get_jwks(team_domain: str, fetcher: Optional[JwksFetcher]) -> dict:
54         """Return the JWKS for ``team_domain``, cached with a TTL. An injected ``fetcher``
(tests)
55         bypasses the cache so each test is deterministic."""
56         if fetcher is not None:
57             return fetcher(team_domain)
58         now = time.time()
59         cached = _jwks_cache.get(team_domain)
60         if cached and (now - cached[0]) < _JWKS_TTL_SEC:
61             return cached[1]
62         jwks = _default_jwks_fetcher(team_domain)
63         _jwks_cache[team_domain] = (now, jwks)
64         return jwks
65
66
67     def verify_cf_access_token(token: str, *, team_domain: str, aud: str, jwks: dict) ->
str:
68         """Verify a Cf-Access JWT against ``jwks`` (RS256; checks sig + exp + issuer +
audience) and
69         return the user_id (``email`` else ``sub``). Raises ``EdgeAuthError`` on ANY
failure."""
70         try:
71             import jwt
72             from jwt import PyJWKSet
73             except Exception as exc: # pragma: no cover - only when the auth extra isn't
installed
74                 raise EdgeAuthError(
75                     "PyJWT[crypto] is required for edge auth ? install the 'auth' extra "
76                     "(pip install -e .[auth]) on a host with edge_auth_enabled."
77                 ) from exc
78
79         try:
80             kid = jwt.get_unverified_header(token).get("kid")
81             if not kid:
82                 raise EdgeAuthError("token header has no kid")
83             signing_key = PyJWKSet.from_dict(jwks)[kid]
84             claims = jwt.decode(
85                 token, signing_key.key, algorithms=["RS256"],
86                 audience=aud, issuer=_issuer(team_domain),
87                 options={"require": ["exp", "iss", "aud"]},
88             )
89         except EdgeAuthError:
90             raise
91         except KeyError as exc: # kid not in the JWKS
92             raise EdgeAuthError(f"no JWKS key for kid {exc}") from exc
93         except Exception as exc: # jwt.* errors: bad sig / expired / wrong aud|iss /
malformed
94             raise EdgeAuthError(f"Cf-Access token verification failed: {exc}") from exc
95
96         user = claims.get("email") or claims.get("sub")
97         if not user:
98             raise EdgeAuthError("verified token carries no email/sub claim")
99         return str(user)
100
101
102     def resolve_tenant(headers: Mapping[str, str], config: Any, *,
103                       jwks_fetcher: Optional[JwksFetcher] = None) -> Optional[str]:
104         """Resolve the tenant ``user_id`` for a request, or ``None`` when edge auth is OFF.
105
106         DEFAULT-OFF (``security.edge_auth_enabled=False``) ? ``None`` (the local single-
user; unchanged).
107         Enabled ? require + verify the Cf-Access header; raise ``EdgeAuthError`` (? 401/403)
on any miss.

```



```

108     ``jwks_fetcher`` is injectable for offline tests.
109     """
110     sec = getattr(config, "security", None)
111     if not sec or not getattr(sec, "edge_auth_enabled", False):
112         return None # DEFAULT-OFF ? no tenant identity, byte-identical to today
113
114     team_domain = getattr(sec, "cf_team_domain", "") or ""
115     aud = getattr(sec, "cf_access_aud", "") or ""
116     if not team_domain or not aud:
117         raise EdgeAuthError(
118             "edge_auth_enabled but security.cf_team_domain / cf_access_aud are not
configured")
119
120     # Headers are case-insensitive over HTTP; Starlette's Headers handles that, but
support a plain
121     # dict too (tests) by trying both casings.
122     token = headers.get(CF_ACCESS_HEADER) or headers.get(CF_ACCESS_HEADER.lower())
123     if not token:
124         raise EdgeAuthError(f"missing {CF_ACCESS_HEADER} header (request did not pass
Cloudflare Access)")
125
126     jwks = _get_jwks(team_domain, jwks_fetcher)
127     return verify_cf_access_token(token, team_domain=team_domain, aud=aud, jwks=jwks)
128

```

7. user (2026-06-21T14:59:27.020Z)

```

280     class SecurityConfig(BaseModel):
281         # When set, /api/process video_path must resolve under this directory (hosted/tenant
282         # mode). Default None preserves the single-user local 'any local file' workflow.
283         allowed_video_root: Optional[str] = None
284
285         # --- Chunked upload (B2, PR-2) ---
286         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
287         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
288         # contain upload_dir or /api/process will reject the uploaded paths.
289         upload_dir: str = "output/uploads"
290         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
291         # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
292         max_upload_mb: int = 4096
293         max_part_mb: int = 95
294         allowed_upload_extensions: List[str] = ["mp4", "mov"]
295
296         # --- M9 edge auth (Cloudflare Access, D9) ---
297         # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
298         # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
Jwt-Assertion`
299         # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
300         # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
301         # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
302         # application's audience tag. NO secrets here ? these are public identifiers.
303         edge_auth_enabled: bool = False
304         cf_team_domain: str = "" # e.g. https://<team>.cloudflareaccess.com (issuer +
/certs JWKS)
305         cf_access_aud: str = "" # the CF Access application AUD tag the token must
carry
306         # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at
import in
307         # backend/main.py so importing the app does no filesystem I/O; default [""].
Credentials stay
308         # OFF ? Cf-Access is a header, not a cookie.)

```

8. assistant (2026-06-21T14:59:29.410Z)

Now let me read the backend/main.py to see how the exposure checks are integrated, and the test file to understand test coverage:

9. user (2026-06-21T14:59:30.638Z)

```

562
563
564     def _enforce_startup_or_exit(config) -> None:
565         """M5: run the startup checks for the server path and ``sys.exit(1)`` on a FATAL
finding BEFORE
566         any work ? a fatal incoherent profile (e.g. cloud-serving pointed at local-only
storage) OR, for
567         the exposed server (``include_exposure=True``, M9 hardening), a half-configured
exposure (edge
568         auth on without the ``allowed_video_root`` jail / CF team+AUD). A strict no-op when
no
569         ``deployment.profile`` is selected AND edge auth is off (today's behaviour). No GPU
probe here ?
570         the server stays torch-light; the detector healthcheck is the GPU authority.
Extracted so the
571         fail-fast wiring + the default-OFF no-op are unit-testable."""
572         try:
573             enforce_startup_checks(config, logger=logger, include_exposure=True)
574         except StartupCheckError as e:
575             logger.error("[STARTUP] refusing to start: %s", e)
576             sys.exit(1)
577
578
579     def main():
580         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
581         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")

```

10. user (2026-06-21T14:59:31.279Z)

```

294         owner_id = edge_auth.resolve_tenant(request.headers, global_config)
295     except edge_auth.EdgeAuthError as e:
296         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
297
298     allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
299     try:
300         validate_input_path(req.video_path, allowed_root=allowed_root)
301         # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
302         # scheme, which we map to a clean 400 (an unknown scheme must never 500).
303         input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
304     except ValueError as e:
305         raise HTTPException(status_code=400, detail=str(e)) from e
306         # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
307         # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
308         # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
309         # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
310         # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)

```

```

311         # rejects a non-local URI as out-of-root.
312         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
313             raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")

```

11. user (2026-06-21T14:59:32.224Z)

```

238         """A warning-only profile (truly-local + cloud storage) must NOT abort the
server."""
239         import backend.main as bmain
240         assert bmain._enforce_startup_or_exit(_cfg(profile="truly-local", backend="gcs")) is
None
241
242
243         # ----- exposure-safety posture (M9 hardening,
server-only)
244
245         def test_exposure_off_by_default_is_noop():
246             """edge_auth_enabled=False (today's default) ? ZERO exposure checks, even with
include_exposure
247             and an otherwise-unsafe config (no allowed_video_root). The strict default-OFF
guarantee."""
248             cfg = _exposed_cfg(edge_auth=False, video_root=None, team="", aud="")
249             assert run_startup_checks(cfg, include_exposure=True, env={}) == []
250
251
252         def test_exposure_not_run_without_flag(monkeypatch):
253             """The worker path (include_exposure=False, the default) NEVER runs exposure checks
? even with
254             edge auth ON and an unsafe config. Only the HTTP server opts in."""
255             monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
256             cfg = _exposed_cfg(edge_auth=True, video_root=None, team="", aud="")
257             assert run_startup_checks(cfg, env={}) == [] # include_exposure defaults False
258
259
260         def test_exposed_without_video_root_is_fatal(monkeypatch):
261             monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
262             cfg = _exposed_cfg(video_root=None) # team+aud set, jail missing
263             checks = run_startup_checks(cfg, include_exposure=True, env={})
264             assert _codes(checks) == {"EXPOSED_NO_VIDEO_ROOT"}
265             assert checks[0].level == LEVEL_ERROR
266
267
268         @pytest.mark.parametrize("video_root", [None, "", " "])
269         def test_exposed_blank_video_root_is_fatal(monkeypatch, video_root):
270             """None, empty, and whitespace-only roots are all treated as 'unset' ? fatal."""
271             monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
272             cfg = _exposed_cfg(video_root=video_root)
273             assert "EXPOSED_NO_VIDEO_ROOT" in _codes(run_startup_checks(cfg,
include_exposure=True, env={}))
274
275
276         @pytest.mark.parametrize("team,aud,expect_missing", [
277             ("", "abc", "cf_team_domain"),
278             ("https://t.cloudfareaccess.com", "", "cf_access_aud"),
279             ("", "", "cf_team_domain + cf_access_aud"),
280         ])
281         def test_exposed_incomplete_cf_config_is_fatal(monkeypatch, team, aud, expect_missing):
282             monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
283             cfg = _exposed_cfg(team=team, aud=aud) # jail set, CF config partial
284             checks = run_startup_checks(cfg, include_exposure=True, env={})
285             assert _codes(checks) == {"EXPOSED_EDGE_AUTH_INCOMPLETE"}
286             assert checks[0].level == LEVEL_ERROR
287             assert expect_missing in checks[0].message # names exactly what's missing
288

```

```

289
290 def test_exposed_missing_auth_extra_warns(monkeypatch):
291     """PyJWT not importable ? a WARN (never fatal ? the auth module's ImportError is the
authority)."""
292     monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: False)
293     cfg = _exposed_cfg() # fully configured otherwise
294     checks = run_startup_checks(cfg, include_exposure=True, env={})
295     assert _codes(checks) == {"EXPOSED_AUTH_EXTRA_MISSING"}
296     assert checks[0].level == LEVEL_WARN
297
298
299 def test_exposed_fully_configured_is_clean(monkeypatch):
300     monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
301     cfg = _exposed_cfg() # edge auth on, jail set, team+aud set, auth extra present
302     assert run_startup_checks(cfg, include_exposure=True, env={}) == []
303
304
305 def test_exposed_both_errors_together(monkeypatch):
306     """An unsafe exposure trips BOTH fatal checks at once (jail missing + CF config
missing)."""
307     monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
308     cfg = _exposed_cfg(video_root=None, team="", aud="")
309     codes = _codes(run_startup_checks(cfg, include_exposure=True, env={}))
310     assert codes == {"EXPOSED_NO_VIDEO_ROOT", "EXPOSED_EDGE_AUTH_INCOMPLETE"}
311
312
313 def test_exposure_composes_with_profile_checks(monkeypatch):
314     """Exposure posture is INDEPENDENT of deployment.profile ? an exposed cloud-serving
config with
315     local storage trips both the exposure jail error AND the profile STORAGE_INCOHERENT
error."""
316     monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
317     cfg = _exposed_cfg(video_root=None, profile="cloud-serving", backend="local")
318     codes = _codes(run_startup_checks(cfg, cuda_available=True, include_exposure=True,
319                                     env={"K_SERVICE": "x"}))
320     assert codes == {"EXPOSED_NO_VIDEO_ROOT", "STORAGE_INCOHERENT"}
321
322
323 def test_server_enforce_exits_on_unsafe_exposure(monkeypatch):
324     """The server's fail-fast wiring aborts (exit 1) when edge auth is on but the engine
is exposed
325     without the allowed_video_root jail ? BEFORE serving a single request."""
326     monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
327     import backend.main as bmain
328     with pytest.raises(SystemExit) as ei:
329         bmain._enforce_startup_or_exit(_exposed_cfg(video_root=None))
330     assert ei.value.code == 1
331
332
333 def test_server_enforce_allows_safe_exposure(monkeypatch):
334     """A fully-configured exposure starts cleanly (no exit)."""
335     monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
336     import backend.main as bmain
337     assert bmain._enforce_startup_or_exit(_exposed_cfg()) is None
338
339
340 def test_worker_path_ignores_unsafe_exposure(monkeypatch, tmp_path):
341     """The worker (cmd path) does NOT enforce exposure posture ? an edge-auth-on config
that would
342     abort the server runs fine on the worker (it reads --video-uri, not untrusted tenant
paths)."""
343     monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
344     from backend.worker import __main__ as wmain
345     cfg_file = tmp_path / "c.json"
346     cfg_file.write_text('{"security": {"edge_auth_enabled": true}}') # exposed-ish, no

```

```
jail/team/aud
347         assert wmain._run_startup_checks(wmain.load_config(str(cfg_file))) is True
```

12. assistant (2026-06-21T14:59:34.288Z)

Now let me check the worker path to see how `_run_startup_checks` is used:

13. user (2026-06-21T14:59:35.275Z)

Found 1 file
backend\worker__main__.py

14. user (2026-06-21T14:59:35.925Z)

```
1      """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3      python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4          [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5      python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7      The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
8      ``local://``
9      is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
10     local runs
11     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
12     ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
13     key) ?
14     the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
15
16     This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
17     CLI uses
18     (validator registry ? segmenter ? report), never a forked pipeline.
19     """
20     from __future__ import annotations
21
22     import argparse
23     import csv
24     import json
25     import logging
26     import os
27     import sys
28     import tempfile
29     from typing import Any, List
30
31     from backend import storage
32     from backend.compute import ComputeJobSpec, get_compute_backend
33     from backend.config import (
34         StartupCheckError,
35         enforce_startup_checks,
36         load_config,
37         probe_cuda_available,
38     )
39     from backend.infrastructure.database import Database
40     from backend.pipeline.segmenters.base import segmenter_registry
41     from backend.pipeline.validators.base import registry as validator_registry
42     from backend.reporting import emit_indexer_report
43     from backend.results import Failure
44     from backend.utils.hashing import compute_video_id
45
46     logger = logging.getLogger("backend.worker")
47
48     def _coerce(v: str) -> Any:
49         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
```

```

str."""
47         low = v.lower()
48         if low in ("true", "false"):
49             return low == "true"
50         for cast in (int, float):
51             try:
52                 return cast(v)
53             except ValueError:
54                 pass
55         return v
56
57
58     def _apply_overrides(config: Any, sets: List[str]) -> None:
59         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
60         for kv in sets or []:
61             key, sep, val = kv.partition("=")
62             if not sep:
63                 raise ValueError(f"--set expects k=v, got {kv!r}")
64             parts = key.split(".")
65             obj = config
66             for p in parts[:-1]:
67                 obj = getattr(obj, p)
68             setattr(obj, parts[-1], _coerce(val))
69
70
71     def _write_intervals_csv(segments: List[dict], path: str) -> None:
72         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
73         artifact (same schema as the rally-annotator labels)."""
74         with open(path, "w", newline="") as f:
75             w = csv.writer(f)
76             w.writerow(["start", "end", "shot_count", "ending_reason"])
77             for s in segments:
78                 w.writerow([
79                     f'{float(s.get("start_time", 0.0)):.3f}',
80                     f'{float(s.get("end_time", 0.0)):.3f}',
81                     s.get("shot_count", ""),
82                     s.get("ending_reason", s.get("shot_count_confidence", "")),
83                 ])
84
85
86     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
87         with open(path, "w") as f:
88             json.dump({
89                 "video_id": video_id,
90                 "status": status,
91                 "segment_count": len(segments),
92                 "segments": [{"start": float(s.get("start_time", 0.0)),
93                             "end": float(s.get("end_time", 0.0))} for s in segments],
94             }, f, indent=2)
95
96
97     def _run_startup_checks(config: Any) -> bool:
98         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
99         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
100         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op

```

15. assistant (2026-06-21T14:59:37.894Z)

Let me continue reading the worker's `_run_startup_checks`:

16. user (2026-06-21T14:59:38.968Z)

```
97     def _run_startup_checks(config: Any) -> bool:
98         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
99         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
100         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
101         (returns True, ZERO work) when no deployment.profile is selected.
102
103         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
104         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
105         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
106         no-op of work, not just of output."""
107         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
108         cuda = probe_cuda_available() if profile else None # torch-free on the default-OFF
path
109         try:
110             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
111         return True
```

17. assistant (2026-06-21T14:59:41.624Z)

Perfect! Now I can see that the worker calls `enforce_startup_checks(config, ...)` without the `include_exposure=True` flag. Let me verify the rest of the context and look at the docs entry more carefully:

18. user (2026-06-21T14:59:42.680Z)

```
165     - 2026-06-21 ? **REAL M7 cloud GPU run ?** (owner-authorized, within the D17 cap):
provisioned an **L4** (`g2-standard-4`, asia-southeast1-a; Mumbai stocked-out) ? native WASB on
the L4 ? **22 rallies** on the smoke clip ?
`gs://?/outputs/05e0e577?/{intervals.csv,report.json}`; VM DELETED ($0, ~$0.21). ? the no-Gemini
cloud run needs `--set indexing.skip_ai_handoff=true` (the fully-local path emits WASB candidate
windows directly ? the long-flagged "0 rallies" cold-start fix). The reel **stitch** validated
locally (`VideoCompiler` ? 22-segment highlights, 85 MB). DEPLOY_REPORT runlog = the cloud-GPU
dry-run guide (`docs/CLOUD_GPU_DRYRUN_GUIDE.md`).
166     - 2026-06-21 ? **M9-auth code landed** ([#290](https://github.com/Khelsutra/badminton-
highlight-indexer/pull/290)): `backend/api/auth.py` ? Cloudflare Access **Cf-Access JWT verify**
(RS256: sig+exp+iss+aud against the team JWKS; lazy `PyJWT[crypto]`; injectable JWKS ? fully-
offline tests) + `resolve_tenant` ? tags `create_job(owner_id=?)`;
`SecurityConfig` {`edge_auth_enabled=False, cf_team_domain, cf_access_aud`} + CORS via
`RALLY_CORS_ALLOW_ORIGINS`. **Default-OFF.** Security review (`wf_9b4c71d0`) found NO bypass;
locked alg=none + HS256-confusion rejection. **? M9 ? ? the consent cols + ToS/DPA are
owner/counsel; the consent migration is now v7 (M8 took v6).**
167     - 2026-06-21 ? **M8 code landed** ([#288](https://github.com/Khelsutra/badminton-
highlight-indexer/pull/288)): per-job **cost ledger** ? `backend/billing.py` (pure `usage ×
pricebook` math, USD internal / INR display) + a **v6 migration** (NULLABLE cost columns on
`jobs` + a versioned `pricebook` table seeded `placeholder-v0` at $0; additive-NULL, idempotent)
+ `record_job_cost` / `get_job_cost` / `get_video_cost` + `GET /api/jobs/{id}/cost` +
`/api/videos/{id}/cost` + `BillingConfig` (**default-OFF**) + a gated `_maybe_record_job_cost`
hook. Adversarial review (`wf_498b24af`) folded a **major silent-GPU-under-bill** (an unpriced
`gpu_type` is now surfaced, not invisible $0) + the **honest no-op** caveat (usage emission is a
follow-up ? cost 0 until wired) + breakdown reconciliation. Suite green, ruff+mypy clean. **? M8
? ? owner inserts real GCP prices (a new pricebook version) + the worker usage-emission is a
follow-up; placeholder ships $0 = no user-facing change. v6 is the shared bump (M9-consent cols
extend it).**
168     - 2026-06-21 ? **M6 code landed** ([#287](https://github.com/Khelsutra/badminton-
highlight-indexer/pull/287)): `backend/compute/` ComputeBackend registry ? `CloudRunCompute`
dispatches a Cloud Run **Job** (injectable client) then **bucket-polls** a done-marker via
`execution_policy.poll_until` + the storage layer (D16); `get_compute_backend` returns a backend
```

ONLY for `compute\_target=cloud\_run` (else `None` ? the worker's in-process path, **default-OFF**). The dispatched container runs **inline** (forced `local\_gpu`+native, never re-dispatch); the worker writes a `--done-uri` completion marker on success.

`deploy/cloudrun/{Dockerfile,README}` = the L4 image (CUDA+ffmpeg+fonts+`wasb` conda env; weights staged per WASB-C3) + the M7 runbook. Adversarial review (`wf\_87b22fa4`) folded a **blocker** (cloud `fetch` needs a dst ? reads silently returned `{}` on gcs/s3) + a poll-wait test gap + `PolicyTimeout`?clean rc 4. Suite green, ruff+mypy clean. **? M6 ? ? the real build/push/L4 run + DEPLOY_REPORT is M7 (owner, D17 budget).**

169 - 2026-06-21 ? **M5 code landed** ([#286](https://github.com/Khelsutra/badminton-highlight-indexer/pull/286)): `backend/config/startup.py` ? per-profile fail/warn-fast startup checks (cloud-serving + non-cloud storage = the one fatal coherence error; GPU/creds mismatches = warnings, the runner healthcheck stays the authority), wired into the server (`\_enforce\_startup\_or\_exit` ? exit 1) + the worker (`cmd\_infer`/`cmd\_train` ? rc 2), default-OFF (probe gated behind an active profile ? the worker stays torch-free at `profile=""`). **L4 profile-parity test** (`tests/test\_profile\_parity.py`): same stub trajectory ? byte-identical windows + segment ranges across truly-local / cpu-mock / no-profile + worker-vs-in-process. Adversarial 4-lens review folded (`wf\_34671ac1`). Suite green, ruff+mypy clean. **? M5 ? ? the real-GPU truly-local runlog (L5) is owner-side** (template pre-staged in `runlogs/`).

19. assistant (2026-06-21T14:59:44.672Z)

Now let me look at the new changelog entry:

20. user (2026-06-21T14:59:45.798Z)

```
diff --git a/docs/COMPUTE_DECOUPLED_SERVING/README.md b/docs/COMPUTE_DECOUPLED_SERVING/README.md
index 3b3b1c5..f59f5d9 100644
--- a/docs/COMPUTE_DECOUPLED_SERVING/README.md
+++ b/docs/COMPUTE_DECOUPLED_SERVING/README.md
@@ -121,7 +121,7 @@ Legend: ? Todo · ? In progress · ? Done · ? Gated. **One small PR per
 | M6 | Cloud Run GPU **dispatch shim** (control plane ? job via storage ? re-claim) +
 **containerized worker image** (ffmpeg+CUDA+WASB+fonts); mocked-dispatch tests | M5 | ? owner
 (GCP spend) | ? | [#287](https://github.com/Khelsutra/badminton-highlight-indexer/pull/287)
 (shim+image+mocked ?; real build/run = M7 ? owner) |
 | M7 | **cloud-serving profile** e2e on L4: upload<2GB` + gdrive/bucket ? detect+stitch on
 Cloud Run ? reel via signed URL; **DEPLOY_REPORT** (posterity, sample runs + contact sheet) | M6
 | ? owner (spend) | ? | ? |
 | M8 | Per-job **cost ledger** (v6 migration: `owner_id, gpu_active_seconds, wall_seconds,
 bytes_out, cost_usd, cost_breakdown_json`) + **pricebook** + aggregation + `/api/.../cost` | M7
 | ? owner (billing) | ? | [#288](https://github.com/Khelsutra/badminton-highlight-
 indexer/pull/288) (ledger+pricebook+/cost ? default-OFF; real prices + usage-emission ?
 owner/follow-up) |
 -| M9 | **Edge auth** (Cf-Access extraction) + per-tenant scoping on `user_id`; DPA/consent gate
 wiring | M7 | ? owner (privacy/counsel) | ? | [#290](https://github.com/Khelsutra/badminton-
 highlight-indexer/pull/290) (auth: Cf-Access JWT verify + owner_id tag, default-OFF ?; consent
 cols + ToS/DPA ? owner/counsel) |
 +| M9 | **Edge auth** (Cf-Access extraction) + per-tenant scoping on `user_id`; DPA/consent gate
 wiring | M7 | ? owner (privacy/counsel) | ? | [#290](https://github.com/Khelsutra/badminton-
 highlight-indexer/pull/290) (auth: Cf-Access JWT verify + owner_id tag, default-OFF ?; **+
 exposure-safety boot posture** in `startup.py` ? edge-auth-on requires the `allowed_video_root`
 jail + CF team/AUD or the server refuses to start, see changelog; consent cols + ToS/DPA ?
 owner/counsel) |
 | M10 | `byo_worker` / zero-infra-BYO ? **design-only, DEFERRED** | M8,M9 | ? explicit owner
 approval | ? | ? |
 | **M11** | **Data-flywheel harness (D12):** on a consented adult job, **capture** the upload +
 Gemini reel/rally output ? **reliably store** under the per-video workspace ? **shadow-eval**
 owned-vs-Gemini (dual-eval-set / `experiment.compare`) ? **promote** good ones to the golden
 TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the owned model
 climbs to the D11 floor. Reuses `data_pipeline/` + `backend/eval/` + the consent gate (M9). |
 M7,M9 | ? owner (consent/privacy) | ? | ? |
 | **M12** | **`gdrive-api` (OAuth) StorageBackend (D14 Northstar):** cloud reads the source
 from + writes the stitched reel **back to the user's Google Drive next to the source** (per-user
 OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local mount
 backend; slots into the `StorageBackend` ABC. | M7,M9 | ? owner (OAuth/consent) | ? | ? |
@@ -167,3 +167,4 @@ Legend: ? Todo · ? In progress · ? Done · ? Gated. **One small PR per
```


- 2026-06-21 ? **M8 code landed** ([#288](https://github.com/Khelsutra/badminton-highlight-indexer/pull/288)): per-job **cost ledger** ? `backend/billing.py` (pure `usage × pricebook` math, USD internal / INR display) + a **v6 migration** (NULLABLE cost columns on `jobs` + a versioned `pricebook` table seeded `placeholder-v0` at \$0; additive-NULL, idempotent) + `record\_job\_cost`/`get\_job\_cost`/`get\_video\_cost` + GET /api/jobs/{id}/cost + /api/videos/{id}/cost + `BillingConfig` (**default-OFF**) + a gated `\_maybe\_record\_job\_cost` hook. Adversarial review (`wf\_498b24af`) folded a **major silent-GPU-under-bill** (an unpriced `gpu\_type` is now surfaced, not invisible \$0) + the **honest no-op** caveat (usage emission is a follow-up ? cost 0 until wired) + breakdown reconciliation. Suite green, ruff+mypy clean. **M8 ? ? owner inserts real GCP prices (a new pricebook version) + the worker usage-emission is a follow-up; placeholder ships \$0 = no user-facing change. v6 is the shared bump (M9-consent cols extend it). **

- 2026-06-21 ? **M6 code landed** ([#287](https://github.com/Khelsutra/badminton-highlight-indexer/pull/287)): `backend/compute/` ComputeBackend registry ? `CloudRunCompute` dispatches a Cloud Run **Job** (injectable client) then **bucket-polls** a done-marker via `execution\_policy.poll\_until` + the storage layer (D16); `get\_compute\_backend` returns a backend ONLY for `compute\_target=cloud\_run` (else `None` ? the worker's in-process path, **default-OFF**). The dispatched container runs **inline** (forced `local\_gpu+native`, never re-dispatch); the worker writes a `--done-uri` completion marker on success. `deploy/cloudrun/{Dockerfile,README}` = the L4 image (CUDA+ffmpeg+fonts+`wasb` conda env; weights staged per WASB-C3) + the M7 runbook. Adversarial review (`wf\_87b22fa4`) folded a **blocker** (cloud `fetch` needs a dst ? reads silently returned `{}` on gcs/s3) + a poll-wait test gap + `PolicyTimeout`?clean rc 4. Suite green, ruff+mypy clean. **M6 ? ? the real build/push/L4 run + DEPLOY_REPORT is M7 (owner, D17 budget). **

- 2026-06-21 ? **M5 code landed** ([#286](https://github.com/Khelsutra/badminton-highlight-indexer/pull/286)): `backend/config/startup.py` ? per-profile fail/warn-fast startup checks (cloud-serving + non-cloud storage = the one fatal coherence error; GPU/creds mismatches = warnings, the runner healthcheck stays the authority), wired into the server (`\_enforce\_startup\_or\_exit` ? exit 1) + the worker (`cmd\_infer`/`cmd\_train` ? rc 2), default-OFF (probe gated behind an active profile ? the worker stays torch-free at `profile=""`). **L4 profile-parity test** (`tests/test\_profile\_parity.py`): same stub trajectory ? byte-identical windows + segment ranges across truly-local / cpu-mock / no-profile + worker-vs-in-process. Adversarial 4-lens review folded (`wf\_34671ac1`). Suite green, ruff+mypy clean. **M5 ? ? the real-GPU truly-local runlog (L5) is owner-side** (template pre-staged in `runlogs/`).

+ - 2026-06-21 ? **M9 hardening: exposure-safety boot posture** (alpha-shareable "tunnel-to-a-box" enabler): extended `backend/config/startup.py` with a **server-only exposure posture** (new `include\_exposure=True` from `\_enforce\_startup\_or\_exit`; the worker stays unaffected ? it reads `--video-uri`, not untrusted tenant paths). Gated on `security.edge\_auth\_enabled` (**default-OFF ? strict no-op**, byte-identical to today): when the owner flips edge auth ON to expose the engine behind the CF-Access gate, three checks fire at **boot** ? `EXPOSED\_NO\_VIDEO\_ROOT` (**fatal**: `allowed\_video\_root` jail unset ? a tenant could read an arbitrary local file), `EXPOSED\_EDGE\_AUTH\_INCOMPLETE` (**fatal**: CF team-domain/AUD missing ? lifts the current first-request 401 to boot), `EXPOSED\_AUTH\_EXTRA\_MISSING` (**warn**: `PyJWT[crypto]` not importable ? would 500 per request). So a half-configured exposure fails fast instead of leaking/500-ing. +13 unit tests (default-OFF no-op, worker-ignores, both-fatals-compose-with-profile-checks, server-exit). Pairs with the appliance **T3** network posture (raw `:8000` refused ? owner-run). Suite green, ruff+mypy clean. **backstops M9 ? the actual exposure flip + CF team/AUD/ingress + key rotation stay owner-side. **

21. assistant (2026-06-21T14:59:47.710Z)

Now let me check the resolve_tenant function more carefully and look for any messaging drift:

22. user (2026-06-21T14:59:49.276Z)

```
3:**DEFAULT-OFF:** when ``security.edge_auth_enabled`` is False the resolver is a strict NO-OP
  (returns
76:         "(pip install -e .[auth]) on a host with edge_auth_enabled."
106:     DEFAULT-OFF (``security.edge_auth_enabled=False``) ? ``None`` (the local single-user;
  unchanged).
111:     if not sec or not getattr(sec, "edge_auth_enabled", False):
114:         team_domain = getattr(sec, "cf_team_domain", "") or ""
115:         aud = getattr(sec, "cf_access_aud", "") or ""
118:         "edge_auth_enabled but security.cf_team_domain / cf_access_aud are not
  configured")
```

23. user (2026-06-21T14:59:50.476Z)

No matches found

24. assistant (2026-06-21T14:59:52.411Z)

Now let me run the full test suite to see test results:

25. user (2026-06-21T15:00:02.516Z)

```
===== test session starts =====
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 43 items

tests/test_startup_checks.py::test_no_profile_is_strict_noop[local-cuda-auto] PASSED [ 2%]
tests/test_startup_checks.py::test_no_profile_is_strict_noop[gcs-cpu-stub] PASSED [ 4%]
tests/test_startup_checks.py::test_no_profile_is_strict_noop[s3-auto-native] PASSED [ 6%]
tests/test_startup_checks.py::test_enforce_no_profile_never_raises_and_returns_empty PASSED [ 9%]
tests/test_startup_checks.py::test_truly_local_local_storage_gpu_present_is_clean PASSED [ 11%]
tests/test_startup_checks.py::test_truly_local_cuda_strict_without_gpu_warns PASSED [ 13%]
tests/test_startup_checks.py::test_truly_local_auto_without_gpu_warns_cpu_fallback PASSED [ 16%]
tests/test_startup_checks.py::test_truly_local_unprobed_cuda_skips_gpu_checks PASSED [ 18%]
tests/test_startup_checks.py::test_truly_local_cloud_storage_warns_unusual PASSED [ 20%]
tests/test_startup_checks.py::test_truly_local_gdrive_storage_is_fine PASSED [ 23%]
tests/test_startup_checks.py::test_cloud_serving_local_storage_is_fatal PASSED [ 25%]
tests/test_startup_checks.py::test_cloud_serving_gcs_without_creds_warns PASSED [ 27%]
tests/test_startup_checks.py::test_cloud_serving_gcs_with_creds_is_clean PASSED [ 30%]
tests/test_startup_checks.py::test_cloud_serving_on_cloud_run_env_skips_creds_warn PASSED [ 32%]
tests/test_startup_checks.py::test_cloud_serving_cpu_device_warns PASSED [ 34%]
tests/test_startup_checks.py::test_cloud_serving_no_gpu_warns PASSED [ 37%]
tests/test_startup_checks.py::test_cpu_mock_stub_is_clean PASSED [ 39%]
tests/test_startup_checks.py::test_cpu_mock_real_detector_warns PASSED [ 41%]
tests/test_startup_checks.py::test_enforce_raises_on_fatal_and_carries_failures PASSED [ 44%]
tests/test_startup_checks.py::test_enforce_does_not_raise_on_warnings_only PASSED [ 46%]
tests/test_startup_checks.py::test_enforce_raise_on_error_false_returns_errors_without_raising PASSED [ 48%]
tests/test_startup_checks.py::test_worker_cmd_infer_aborts_rc2_on_incoherent_profile PASSED [ 51%]
tests/test_startup_checks.py::test_worker_default_off_skips_cuda_probe PASSED [ 53%]
tests/test_startup_checks.py::test_worker_active_profile_probes_cuda PASSED [ 55%]
tests/test_startup_checks.py::test_server_enforce_exits_on_incoherent_profile PASSED [ 58%]
tests/test_startup_checks.py::test_server_enforce_is_noop_on_default_profile PASSED [ 60%]
tests/test_startup_checks.py::test_server_enforce_does_not_exit_on_warnings PASSED [ 62%]
tests/test_startup_checks.py::test_exposure_off_by_default_is_noop PASSED [ 65%]
tests/test_startup_checks.py::test_exposure_not_run_without_flag PASSED [ 67%]
tests/test_startup_checks.py::test_exposed_without_video_root_is_fatal PASSED [ 69%]
tests/test_startup_checks.py::test_exposed_blank_video_root_is_fatal[None] PASSED [ 72%]
tests/test_startup_checks.py::test_exposed_blank_video_root_is_fatal[] PASSED [ 74%]
tests/test_startup_checks.py::test_exposed_blank_video_root_is_fatal[ ] PASSED [ 76%]
tests/test_startup_checks.py::test_exposed_incomplete_cf_config_is_fatal[-abc-cf_team_domain] PASSED [ 79%]
tests/test_startup_checks.py::test_exposed_incomplete_cf_config_is_fatal[https://t.cloudflareaccess.com--cf_access_aud] PASSED [ 81%]
tests/test_startup_checks.py::test_exposed_incomplete_cf_config_is_fatal[--cf_team_domain + cf_access_aud] PASSED [ 83%]
tests/test_startup_checks.py::test_exposed_missing_auth_extra_warns PASSED [ 86%]
tests/test_startup_checks.py::test_exposed_fully_configured_is_clean PASSED [ 88%]
tests/test_startup_checks.py::test_exposed_both_errors_together PASSED [ 90%]
tests/test_startup_checks.py::test_exposure_composes_with_profile_checks PASSED [ 93%]
```

```
tests/test_startup_checks.py::test_server_enforce_exits_on_unsafe_exposure PASSED [ 95%]
tests/test_startup_checks.py::test_server_enforce_allows_safe_exposure PASSED [ 97%]
tests/test_startup_checks.py::test_worker_path_ignores_unsafe_exposure PASSED [100%]
```

```
===== 43 passed in 7.08s =====
```

26. assistant (2026-06-21T15:00:04.844Z)

Excellent! All tests pass. Now let me conduct a detailed adversarial review by checking for the specific issues mentioned:

27. user (2026-06-21T15:00:07.161Z)

```
===== test session starts =====
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 1 item
```

```
tests/test_startup_checks.py::test_exposed_missing_auth_extra_warns PASSED
```

```
===== 1 passed in 0.32s =====
```

28. assistant (2026-06-21T15:00:09.212Z)

Now let me check if there's an explicit test that WARN does not cause server exit:

29. user (2026-06-21T15:00:20.171Z)

```
20:30:19 [WARNING] backend.main: [STARTUP] EXPOSED_AUTH_EXTRA_MISSING:
security.edge_auth_enabled is ON but PyJWT[crypto] is not importable ? Cf-Access verification
will fail at request time. Install the 'auth' extra (pip install -e .[auth]).
SUCCESS: WARN-only config did not exit
```

30. assistant (2026-06-21T15:00:22.586Z)

Perfect! Now let me check the specific issue: is there duplicated logic for CF config checking between `\_exposure\_checks` and `resolve\_tenant`? This is the message quality concern:

31. user (2026-06-21T15:00:23.563Z)

```
114         # (2) Edge auth cannot verify a token without the team domain + AUD. Today that only
115         # FIRST request (auth.resolve_tenant); lift it to BOOT so a misconfigured gate never
116         # "healthy" then 401s every request.
117         team = (getattr(sec, "cf_team_domain", "") or "").strip()
118         aud = (getattr(sec, "cf_access_aud", "") or "").strip()
119         if not team or not aud:
120             missing = " + ".join(n for n, v in (("cf_team_domain", team), ("cf_access_aud",
121             aud)) if not v)
122             out.append(StartupCheck(
123                 LEVEL_ERROR, "EXPOSED_EDGE_AUTH_INCOMPLETE",
124                 f"security.edge_auth_enabled is ON but {missing} is not configured ? the Cf-
125                 Access JWT "
126                 "cannot be verified (every request would fail). Set the CF team domain +
127                 Access AUD.",
128             ))
```

32. user (2026-06-21T15:00:24.524Z)

```

114     team_domain = getattr(sec, "cf_team_domain", "") or ""
115     aud = getattr(sec, "cf_access_aud", "") or ""
116     if not team_domain or not aud:
117         raise EdgeAuthError(
118             "edge_auth_enabled but security.cf_team_domain / cf_access_aud are not
configured")
119
120     # Headers are case-insensitive over HTTP; Starlette's Headers handles that, but
support a plain
121     # dict too (tests) by trying both casings.

```

33. assistant (2026-06-21T15:00:27.759Z)

I found a potential drift issue! Let me check the logic more carefully. In `auth.py` line 114-115, they do `.strip()` only in the `or ""` fallback, but in `startup.py` lines 117-118 they explicitly `.strip()` on the strings. Let me verify this logic:

34. user (2026-06-21T15:00:32.629Z)

```

team=''                                aud='abc'                                auth_valid=False
startup_valid=False MATCH=True
team=' '                                aud='abc'                                auth_valid=True
startup_valid=False MATCH=False
team='https://t.cloudflareaccess.com' aud=''                                auth_valid=False
startup_valid=False MATCH=True
team='https://t.cloudflareaccess.com' aud=' '                                auth_valid=True
startup_valid=False MATCH=False

```

35. assistant (2026-06-21T15:00:36.361Z)

****FOUND A REAL BUG!**** The logic differs. In `auth.py`, whitespace-only strings (e.g., `""`) are treated as valid (because `""` or `""` evaluates to `""`), but in `startup.py`, they're treated as invalid (because `.strip()` makes them empty). This is a divergence that could cause the server to start but then immediately fail on the first request.

Let me verify this is actually a problem in a real scenario:

36. user (2026-06-21T15:00:40.863Z)

```

resolve_tenant raised EdgeAuthError: missing Cf-Access-Jwt-Assertion header (request did not
pass Cloudflare Access)

```

37. assistant (2026-06-21T15:00:43.026Z)

The error comes from missing header, let me test the actual validation with a token:

38. user (2026-06-21T15:00:49.357Z)

```

Test 1 (whitespace team): ['EXPOSED_EDGE_AUTH_INCOMPLETE']
Test 2 (whitespace aud): ['EXPOSED_EDGE_AUTH_INCOMPLETE']

```

39. assistant (2026-06-21T15:00:51.760Z)

Good ? the ****startup check correctly catches**** whitespace. But now let me verify what happens in `auth.resolve\_tenant` with the same whitespace config:

40. user (2026-06-21T15:00:55.613Z)

```

team_domain=' ', aud='abc123'
bool(team_domain)=True, bool(aud)=True
Will check pass? True

```

41. assistant (2026-06-21T15:00:59.915Z)

****CRITICAL BUG FOUND:**** The ``auth.resolve_tenant`` check at line 116 (``if not team_domain or not aud``) would ****pass**** with whitespace-only values, but the startup check would ****reject**** them. This creates a gap: the server starts up (passes the boot check because `startup.py` strips), but then fails on the first request when `auth.resolve_tenant` sees the whitespace-only string is "truthy" but invalid to use.

Actually wait, let me re-read ``auth.py`` more carefully to see if it strips later:

42. user (2026-06-21T15:01:00.854Z)

```
102     def resolve_tenant(headers: Mapping[str, str], config: Any, *,
103                          jwks_fetcher: Optional[JwksFetcher] = None) -> Optional[str]:
104         """Resolve the tenant ``user_id`` for a request, or ``None`` when edge auth is OFF.
105
106         DEFAULT-OFF (``security.edge_auth_enabled=False``) ? ``None`` (the local single-
107         user; unchanged).
108         Enabled ? require + verify the Cf-Access header; raise ``EdgeAuthError`` (? 401/403)
109         on any miss.
110         ``jwks_fetcher`` is injectable for offline tests.
111         """
112         sec = getattr(config, "security", None)
113         if not sec or not getattr(sec, "edge_auth_enabled", False):
114             return None # DEFAULT-OFF ? no tenant identity, byte-identical to today
115
116         team_domain = getattr(sec, "cf_team_domain", "") or ""
117         aud = getattr(sec, "cf_access_aud", "") or ""
118         if not team_domain or not aud:
119             raise EdgeAuthError(
120                 "edge_auth_enabled but security.cf_team_domain / cf_access_aud are not
121                 configured")
122
123         # Headers are case-insensitive over HTTP; Starlette's Headers handles that, but
124         support a plain
125         # dict too (tests) by trying both casings.
126         token = headers.get(CF_ACCESS_HEADER) or headers.get(CF_ACCESS_HEADER.lower())
127         if not token:
128             raise EdgeAuthError(f"missing {CF_ACCESS_HEADER} header (request did not pass
129             Cloudflare Access)")
130
131         jwks = _get_jwks(team_domain, jwks_fetcher)
132         return verify_cf_access_token(token, team_domain=team_domain, aud=aud, jwks=jwks)
```

43. assistant (2026-06-21T15:01:04.061Z)

The check at line 116 in ``auth.py`` will pass with whitespace (because ``" " or "" ? "`` is "truthy"). Then at line 126, it calls ``_get_jwks(team_domain, ...)`` with the whitespace-padded team domain. Let me check if that's a problem:

44. user (2026-06-21T15:01:04.705Z)

```
38     def _issuer(team_domain: str) -> str:
39         return team_domain.rstrip("/")
40
41
42     def _default_jwks_fetcher(team_domain: str) -> dict:
43         """Fetch the team's public JWKS (``<team_domain>/cdn-cgi/access/certs``). Network ?
44         only the
45         production path reaches here; tests pass an injected fetcher."""
46         url = _issuer(team_domain) + _JWKS_PATH
47         with urllib.request.urlopen(url, timeout=5) as resp: # noqa: S310 (https team
48             domain)
49             data: Any = json.load(resp)
50             if not isinstance(data, dict) or "keys" not in data:
```

```

49         raise EdgeAuthError(f"unexpected JWKS shape from {url}")
50     return data
51
52
53     def _get_jwks(team_domain: str, fetcher: Optional[JwksFetcher]) -> dict:
54         """Return the JWKS for ``team_domain``, cached with a TTL. An injected ``fetcher``
(tests)
55         bypasses the cache so each test is deterministic."""
56         if fetcher is not None:
57             return fetcher(team_domain)

```

45. assistant (2026-06-21T15:01:10.260Z)

So if `team\_domain = " "` (whitespace only), then `\_issuer(team\_domain)` will return `".rstrip("/")` = `""`, and the URL will be `"" + "/cdn-cgi/access/certs" = `"/cdn-cgi/access/certs"`, which will fail at runtime. This is a ****DRIFT**** issue:

```

- **startup.py** (line 117): `team = (getattr(sec, "cf_team_domain", "") or "").strip()` ?
whitespace-only is treated as unset (FATAL)
- **auth.py** (line 114): `team_domain = getattr(sec, "cf_team_domain", "") or ""` ? whitespace-
only is treated as SET (will fail at JWKS fetch time with a confusing error)

```

This is a ****MAJOR**** finding. The two code paths have different validation logic, meaning a config that passes startup checks could fail at runtime with a confusing message.

Now let me check if there are any other issues:

46. user (2026-06-21T15:01:19.276Z)

```

tests/test_startup_checks.py::test_exposure_off_by_default_is_noop PASSED [ 16%]
tests/test_startup_checks.py::test_exposure_not_run_without_flag PASSED [ 33%]
tests/test_startup_checks.py::test_exposure_composes_with_profile_checks PASSED [ 50%]
tests/test_startup_checks.py::test_server_enforce_exits_on_unsafe_exposure PASSED [ 66%]
tests/test_startup_checks.py::test_server_enforce_allows_safe_exposure PASSED [ 83%]
tests/test_startup_checks.py::test_worker_path_ignores_unsafe_exposure PASSED [100%]

```

===== 6 passed, 37 deselected in 5.50s =====

47. assistant (2026-06-21T15:01:21.849Z)

Let me verify the default-OFF byte-identity by checking if the worker is affected with edge auth on but no exposure checks:

48. user (2026-06-21T15:01:28.164Z)

```

Server checks (include_exposure=True): ['EXPOSED_NO_VIDEO_ROOT', 'EXPOSED_EDGE_AUTH_INCOMPLETE']
Worker checks (include_exposure=False): []
Byte-identical? False

```

49. assistant (2026-06-21T15:01:30.756Z)

Perfect ? this is correct behavior. The server catches the errors, the worker doesn't (as intended). Now let me review the message quality issue more carefully. Looking at line 120 in startup.py:

50. user (2026-06-21T15:01:31.789Z)

```

114         # (2) Edge auth cannot verify a token without the team domain + AUD. Today that only
fails on the
115         # FIRST request (auth.resolve_tenant); lift it to BOOT so a misconfigured gate never
starts
116         # "healthy" then 401s every request.
117         team = (getattr(sec, "cf_team_domain", "") or "").strip()
118         aud = (getattr(sec, "cf_access_aud", "") or "").strip()

```

```

119         if not team or not aud:
120             missing = " + ".join(n for n, v in (("cf_team_domain", team), ("cf_access_aud",
aud))) if not v)
121             out.append(StartupCheck(
122                 LEVEL_ERROR, "EXPOSED_EDGE_AUTH_INCOMPLETE",
123                 f"security.edge_auth_enabled is ON but {missing} is not configured ? the Cf-
Access JWT "
124                 "cannot be verified (every request would fail). Set the CF team domain +
Access AUD.",
125             ))

```

51. assistant (2026-06-21T15:01:36.791Z)

I see a logic bug at line 120! The variable names in the tuple are backwards. It checks `if not v` (i.e., if the stripped value is falsy), so when the loop runs:

```

- `("cf_team_domain", team)` ? if `team` is empty, it adds `"cf_team_domain"` ?
- `("cf_access_aud", aud)` ? if `aud` is empty, it adds `"cf_access_aud"` ?

```

Actually, that's correct. The logic is: `if not v` means "if the **value** v is falsy (i.e., empty after stripping)", so it's correct.

Now let me check for any test that verifies the error messages don't block the server (only errors do):

52. user (2026-06-21T15:01:37.973Z)

```

237: def test_server_enforce_does_not_exit_on_warnings():

```

53. assistant (2026-06-21T15:01:40.207Z)

Good! There's a test that warnings don't cause exit. Let me also verify there's no lingering reference issue between the two modules:

54. user (2026-06-21T15:01:43.167Z)

startup.py mentions auth

```

Line 13:     own healthcheck is the real authority. A warning surfaces it early without false-
failing.

```

```

Line 16:     we can't probe offline, so the storage backend's own init is the authority; we
just flag it.

```

```

Line 19: behind an edge gate (the owner flips ``security.edge_auth_enabled`` on for the
Cloudflare-Access

```

```

Line 22: rest of the posture at boot: the ``allowed_video_root`` path-jail is set (else an
authenticated

```

```

Line 25: ``auth`` extra is importable (else every request 500s ? warning). Only the HTTP
server passes

```

```

Line 30: a strict no-op ? exactly the pre-M5 behaviour; with ``edge_auth_enabled=False``
(today's default)

```

```

Line 75:     only downgrades to a WARNING, never a hard failure (the storage init is the
authority)."""

```

```

Line 79: def _auth_extra_available() -> bool:

```

```

Line 80:     """True if PyJWT (the ``auth`` extra) is importable ? probed WITHOUT importing it
(find_spec),

```

```

Line 91:     """Exposure-safety posture for an exposed HTTP server. Gated on
``security.edge_auth_enabled``

```

```

Line 92:     (default False ? ``[]`` = strict no-op, today's behaviour). When edge auth is ON
? the "I am

```

```

Line 95:     per request. Config-only (no IO beyond the ``find_spec`` auth-extra probe)."""

```

```

Line 97:     if not sec or not getattr(sec, "edge_auth_enabled", False):

```

```

Line 98:         return [] # DEFAULT-OFF: edge auth off ? no posture enforcement (byte-
identical to today).

```

```

Line 102:     # (1) The path-jail is load-bearing once authenticated tenants can hit
/api/process: WITHOUT it,

```

```

Line 109:          "security.edge_auth_enabled is ON (engine exposed) but
security.allowed_video_root is "
Line 110:          "unset ? an authenticated tenant could make /api/process read an
arbitrary local file. "
Line 114:          # (2) Edge auth cannot verify a token without the team domain + AUD. Today that
only fails on the
Line 115:          # FIRST request (auth.resolve_tenant); lift it to BOOT so a misconfigured gate
never starts
Line 122:          LEVEL_ERROR, "EXPOSED_EDGE_AUTH_INCOMPLETE",
Line 123:          f"security.edge_auth_enabled is ON but {missing} is not configured ? the
Cf-Access JWT "
Line 127:          # (3) The auth extra (PyJWT[crypto]) is lazy-imported by
auth.verify_cf_access_token. If it's
Line 129:          # fatal: the auth module's own ImportError is the final authority.
Line 130:          if not _auth_extra_available():
Line 132:          LEVEL_WARN, "EXPOSED_AUTH_EXTRA_MISSING",
Line 133:          "security.edge_auth_enabled is ON but PyJWT[crypto] is not importable ?
Cf-Access "
Line 134:          "verification will fail at request time. Install the 'auth' extra (pip
install -e .[auth]).",
Line 153:          DEFAULT-OFF: no profile ? no profile checks; ``edge_auth_enabled=False`` ? no
exposure checks.
Line 154:          Pure: no IO (beyond the ``find_spec`` auth probe), no torch import ? the caller
owns the GPU probe.
Line 160:          # runs with no profile set), gated on edge_auth_enabled (default-OFF). Server-
only.
Line 166:          return checks # no profile ? only the exposure posture (itself off unless
edge auth is on).
Line 182:          # GPU heads-up (warning only ? the detector healthcheck is the authority;
the probe is
Line 197:          "can be a false alarm; the runner's DeviceContext/healthcheck is
the authority.",
auth.py does NOT mention startup

```

55. assistant (2026-06-21T15:01:45.591Z)

Good ? they don't cross-import (only comments reference each other). Now let me verify the drift in logic is real by creating a test case:

56. user (2026-06-21T15:01:50.181Z)

```

auth.py: team_domain=' ', would pass check? True
startup.py: team='', would pass check? False

```

MISMATCH: auth.py would PASS, startup.py would FAIL

57. assistant (2026-06-21T15:01:52.363Z)

Perfect confirmation. Now let me write a comprehensive summary and check for any other concerns:

58. user (2026-06-21T15:02:03.307Z)

```

Only jail missing: ['EXPOSED_NO_VIDEO_ROOT']
Only CF config missing: ['EXPOSED_EDGE_AUTH_INCOMPLETE']
Only auth extra missing: ['EXPOSED_AUTH_EXTRA_MISSING']

```

All single-gap scenarios covered ?

59. assistant (2026-06-21T15:02:05.918Z)

Excellent. Now let me verify the edge case where edge_auth is ON + only auth-extra missing (should NOT block server start, only warn):

60. user (2026-06-21T15:02:11.413Z)


```
WARNING:test:[STARTUP] EXPOSED_AUTH_EXTRA_MISSING: security.edge_auth_enabled is ON but
PyJWT[crypto] is not importable ? Cf-Access verification will fail at request time. Install the
'auth' extra (pip install -e .[auth]).
enforce_startup_checks returned checks: ['EXPOSED_AUTH_EXTRA_MISSING']
SUCCESS: WARN-only did not raise
```

61. assistant (2026-06-21T15:02:16.847Z)

Perfect! Now let me verify the test coverage for the exact scenario: does the test suite verify that a WARN (auth-extra missing) does NOT block the server from starting? Let me search for this:

62. user (2026-06-21T15:02:17.747Z)

```
def test_exposed_missing_auth_extra_warns(monkeypatch):
    """PyJWT not importable ? a WARN (never fatal ? the auth module's ImportError is the
    authority)."""
    monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: False)
    cfg = _exposed_cfg() # fully configured otherwise
    checks = run_startup_checks(cfg, include_exposure=True, env={})
    assert _codes(checks) == {"EXPOSED_AUTH_EXTRA_MISSING"}
    assert checks[0].level == LEVEL_WARN
```

```
def test_exposed_fully_configured_is_clean(monkeypatch):
    monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
```

63. assistant (2026-06-21T15:02:20.232Z)

The test verifies the WARN is present, but does NOT test that enforce_startup_checks doesn't raise. Let me check if there's a test for that:

64. user (2026-06-21T15:02:21.176Z)

(Bash completed with no output)

65. assistant (2026-06-21T15:02:23.490Z)

Let me check if there's a test that the server allows a WARN-only exposure:

66. user (2026-06-21T15:02:24.157Z)

```
def test_server_enforce_allows_safe_exposure(monkeypatch):
    """A fully-configured exposure starts cleanly (no exit)."""
    monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
    import backend.main as bmain
```

67. assistant (2026-06-21T15:02:27.330Z)

That test only covers the fully-configured case. There is NO test that explicitly verifies `enforce\_startup\_checks` with a WARN-only exposure (auth extra missing but everything else set) does NOT exit. However, the general test `test\_enforce\_does\_not\_exit\_on\_warnings` should cover this implicitly. Let me verify that test works for exposure warnings too:

68. user (2026-06-21T15:02:29.512Z)

```
===== test session starts =====
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
```

```
collecting ... ERROR: not found: C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_startup_checks.py::test_enforce_does_not_exit_on_warnings
(no match in any of [<Module test_startup_checks.py>])
```

```
collected 0 items
```

```
===== no tests ran in 0.27s =====
```

69. assistant (2026-06-21T15:02:31.246Z)

Let me find the correct test name:

70. user (2026-06-21T15:02:32.226Z)

```
def test_truly_local_cuda_strict_without_gpu_warns():
def test_truly_local_auto_without_gpu_warns_cpu_fallback():
def test_truly_local_cloud_storage_warns_unusual():
def test_cloud_serving_gcs_without_creds_warns():
def test_cloud_serving_on_cloud_run_env_skips_creds_warn():
```

71. assistant (2026-06-21T15:02:34.093Z)

Let me search for the test that enforces warnings don't exit:

72. user (2026-06-21T15:02:34.800Z)

```
61: def test_enforce_no_profile_never_raises_and_returns_empty():
158: def test_enforce_raises_on_fatal_and_carries_failures(caplog):
167: def test_enforce_does_not_raise_on_warnings_only():
173: def test_enforce_raise_on_error_false_returns_errors_without_raising():
223: def test_server_enforce_exits_on_incoherent_profile():
231: def test_server_enforce_is_noop_on_default_profile():
237: def test_server_enforce_does_not_exit_on_warnings():
323: def test_server_enforce_exits_on_unsafe_exposure(monkeypatch):
333: def test_server_enforce_allows_safe_exposure(monkeypatch):
```

73. assistant (2026-06-21T15:02:36.874Z)

Let me check test_enforce_does_not_raise_on_warnings_only:

74. user (2026-06-21T15:02:37.680Z)

```
167     def test_enforce_does_not_raise_on_warnings_only():
168         cfg = _cfg(profile="truly-local", backend="local", device="auto")
169         checks = enforce_startup_checks(cfg, cuda_available=False, env={}) # CPU_FALLBACK
warning
170         assert checks and all(c.level == LEVEL_WARN for c in checks) # returned, not raised
171
172
```

75. assistant (2026-06-21T15:02:41.764Z)

That test is for profile warnings, not exposure warnings. There IS a test gap: ****there is no explicit test that a WARN-only exposure-safety check (e.g., auth-extra missing) doesn't cause server exit via `\_enforce\_startup\_or\_exit`****. The implicit coverage exists through the profile warnings tests, but NOT explicitly for exposure warnings.

Now let me create a comprehensive summary of all findings:

76. user (2026-06-21T15:03:05.863Z)

Structured output provided successfully

Summary

I've completed an adversarial code review of the exposure-safety boot posture changes in the badminton-highlight-indexer. Here are the key findings:

****Blocker (Fix Required)****

****Validation drift between startup.py and auth.py**** ? The two modules handle whitespace-only ``cf_team_domain`` and ``cf_access_aud`` values differently:

- ****startup.py**** (line 117-118): explicitly `.strip()`` the values, treating whitespace-only strings as unset ? ****ERROR at boot****
- ****auth.py**** (line 114-115): does NOT strip, treating whitespace-only strings as valid ? ****fails at JWKS fetch time**** on the first request with a confusing URL error

This creates a gap where a config passes the server startup check but fails on the first request with a misleading error. Both should use `.strip()`` consistently.

****Minor Issues****

1. ****Test gap****: There's no explicit test that a WARN-only exposure check (e.g., auth-extra missing with everything else configured) doesn't block server startup. While implicitly covered by general warning tests, it should be explicitly tested for the exposure posture.
2. ****Message clarity****: The ``EXPOSED_EDGE_AUTH_INCOMPLETE`` error message could be more actionable by explicitly naming the config fields (``security.cf_team_domain``, ``security.cf_access_aud``) instead of just saying "the CF team domain + Access AUD".

****Positive Findings****

- ? All 43 tests pass
- ? Default-OFF guarantee is preserved (byte-identical when `edge_auth_enabled=False`)
- ? Server-vs-worker scoping is correct (worker doesn't run exposure checks)
- ? All three exposure checks are present and tested (jail, CF config, auth-extra)
- ? Both fatal errors properly block server startup; WARN-only allows startup
- ? Good composition with profile checks (both can fire independently)

****Recommendation****: Ship with fixes. Apply the `.strip()`` fix to auth.py and add the missing WARN-only test before merging.